

Spotifry

Grado en Ingeniería Informática - UGR

Generated by Doxygen 1.9.8

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Album Class Reference	5
3.1.1 Detailed Description	7
3.1.2 Constructor & Destructor Documentation	7
3.1.2.1 Album() [1/3]	7
3.1.2.2 Album() [2/3]	7
3.1.2.3 Album() [3/3]	8
3.1.2.4 ~Album()	8
3.1.3 Member Function Documentation	8
3.1.3.1 addSong()	8
3.1.3.2 getArtist()	9
3.1.3.3 getNumSongs()	10
3.1.3.4 getReleaseDate()	10
3.1.3.5 getSong()	10
3.1.3.6 getTitle()	11
3.1.3.7 operator=()	11
3.1.3.8 operator[]()	12
3.1.3.9 setArtist()	12
3.1.3.10 setNewSongs()	12
3.1.3.11 setReleaseDate()	13
3.1.3.12 setTitle()	13
3.2 AlbumList Class Reference	13
3.2.1 Detailed Description	14
3.2.2 Constructor & Destructor Documentation	14
3.2.2.1 AlbumList()	14
3.2.2.2 ~AlbumList()	15
3.2.3 Member Function Documentation	15
3.2.3.1 dump()	15
3.2.3.2 getAlbumById()	17
3.2.3.3 getCurrentSize()	17
3.2.3.4 printAlbum()	17
3.3 Artist Class Reference	19
3.3.1 Detailed Description	19
3.3.2 Constructor & Destructor Documentation	19
3.3.2.1 Artist() [1/2]	19
3.3.2.2 Artist() [2/2]	19
3.3.3 Member Function Documentation	20

3.3.3.1 getName()	20
3.3.3.2 setName()	20
3.4 ArtistList Class Reference	21
3.4.1 Detailed Description	21
3.4.2 Constructor & Destructor Documentation	21
3.4.2.1 ArtistList()	21
3.4.2.2 ~ArtistList()	22
3.4.3 Member Function Documentation	22
3.4.3.1 getArtistById()	22
3.4.3.2 getCurrentSize()	22
3.5 Block Struct Reference	23
3.5.1 Detailed Description	24
3.5.2 Member Data Documentation	25
3.5.2.1 current_size	25
3.5.2.2 list	25
3.5.2.3 next	25
3.6 Date Class Reference	25
3.6.1 Detailed Description	27
3.6.2 Constructor & Destructor Documentation	27
3.6.2.1 Date() [1/2]	27
3.6.2.2 Date() [2/2]	27
3.6.3 Member Function Documentation	28
3.6.3.1 getDay()	28
3.6.3.2 getMonth()	28
3.6.3.3 getYear()	29
3.6.3.4 operator!=(())	29
3.6.3.5 operator<()	29
3.6.3.6 operator<=()	30
3.6.3.7 operator==(())	30
3.6.3.8 operator>()	30
3.6.3.9 operator>=()	31
3.6.3.10 setDay()	31
3.6.3.11 setMonth()	31
3.6.3.12 setYear()	31
3.7 Playlist Class Reference	32
3.7.1 Detailed Description	34
3.7.2 Constructor & Destructor Documentation	34
3.7.2.1 Playlist() [1/3]	34
3.7.2.2 Playlist() [2/3]	35
3.7.2.3 Playlist() [3/3]	35
3.7.2.4 ~Playlist()	35
3.7.3 Member Function Documentation	35

3.7.3.1 addSong()	35
3.7.3.2 deleteSong()	36
3.7.3.3 getCreationDate()	36
3.7.3.4 getCreator()	37
3.7.3.5 getMaxSongs()	37
3.7.3.6 getNumSongs()	38
3.7.3.7 getPrivacy()	38
3.7.3.8 getTitle()	38
3.7.3.9 operator!=(())	39
3.7.3.10 operator=()	39
3.7.3.11 operator==(())	39
3.7.3.12 operator[]()	39
3.7.3.13 setCreationDate()	40
3.7.3.14 setPrivacy()	40
3.7.3.15 setTitle()	41
3.8 Song Class Reference	41
3.8.1 Detailed Description	43
3.8.2 Constructor & Destructor Documentation	43
3.8.2.1 Song() [1/2]	43
3.8.2.2 Song() [2/2]	43
3.8.3 Member Function Documentation	43
3.8.3.1 getDuration()	43
3.8.3.2 getGenre()	44
3.8.3.3 getTitle()	44
3.8.3.4 operator!=(())	45
3.8.3.5 operator==(())	45
3.8.3.6 setDuration()	46
3.8.3.7 setGenre()	46
3.8.3.8 setTitle()	46
3.9 SongList Class Reference	46
3.9.1 Detailed Description	47
3.9.2 Constructor & Destructor Documentation	47
3.9.2.1 SongList()	47
3.9.2.2 ~SongList()	48
3.9.3 Member Function Documentation	48
3.9.3.1 addSong()	48
3.9.3.2 findSong()	49
3.10 Spotify Class Reference	51
3.10.1 Detailed Description	53
3.10.2 Constructor & Destructor Documentation	53
3.10.2.1 Spotify()	53
3.10.3 Member Function Documentation	54

3.10.3.1 artistaTop()	54
3.10.3.2 createPlaylist()	55
3.10.3.3 generarCanciones()	56
3.10.3.4 generarPlaylists()	57
3.10.3.5 getAlbumList()	58
3.10.3.6 getSongList()	59
3.10.3.7 getUserList()	59
3.10.3.8 leerArchivo()	59
3.10.3.9 leerTemplateHtml()	60
3.10.3.10 numPlaylists()	61
3.10.3.11 numSongs()	62
3.10.3.12 reemplazar()	63
3.11 User Class Reference	64
3.11.1 Detailed Description	66
3.11.2 Constructor & Destructor Documentation	67
3.11.2.1 User() [1/3]	67
3.11.2.2 User() [2/3]	67
3.11.2.3 User() [3/3]	67
3.11.2.4 ~User()	67
3.11.3 Member Function Documentation	68
3.11.3.1 addPlaylist()	68
3.11.3.2 addSong()	68
3.11.3.3 createPlaylist()	69
3.11.3.4 deletePlaylist()	70
3.11.3.5 deleteSong()	71
3.11.3.6 getBirthdate()	71
3.11.3.7 getEmail()	71
3.11.3.8 getGender()	72
3.11.3.9 getName()	72
3.11.3.10 getNumPlaylists()	73
3.11.3.11 getNumSavedSongs()	74
3.11.3.12 operator() [1/2]	74
3.11.3.13 operator() [2/2]	74
3.11.3.14 operator=()	75
3.11.3.15 operator[]()	75
3.11.3.16 setBirthdate()	75
3.11.3.17 setEmail()	76
3.11.3.18 setGender()	76
3.11.3.19 setName()	76
3.12 UserList Class Reference	76
3.12.1 Detailed Description	77
3.12.2 Constructor & Destructor Documentation	78

3.12.2.1 UserList()	78
3.12.2.2 ~UserList()	79
3.12.3 Member Function Documentation	79
3.12.3.1 addPlaylist()	79
3.12.3.2 addUser()	79
3.12.3.3 getCurrentSize()	80
3.12.3.4 getNumberOfPlaylists()	80
3.12.3.5 getUserByEmail()	80
4 File Documentation	83
4.1 include/album.h File Reference	83
4.1.1 Detailed Description	84
4.1.2 Function Documentation	84
4.1.2.1 operator<<()	84
4.2 album.h	85
4.3 include/albumlist.h File Reference	86
4.3.1 Detailed Description	88
4.3.2 Variable Documentation	88
4.3.2.1 MAX_ALBUMS	88
4.4 albumlist.h	88
4.5 include/artist.h File Reference	89
4.5.1 Detailed Description	90
4.5.2 Function Documentation	90
4.5.2.1 operator<<()	90
4.6 artist.h	90
4.7 include/artistlist.h File Reference	91
4.7.1 Detailed Description	92
4.7.2 Variable Documentation	92
4.7.2.1 MAX_ARTISTS	92
4.8 artistlist.h	93
4.9 include/date.h File Reference	93
4.9.1 Detailed Description	94
4.9.2 Function Documentation	94
4.9.2.1 operator<<()	94
4.10 date.h	95
4.11 include/playlist.h File Reference	96
4.11.1 Detailed Description	97
4.11.2 Enumeration Type Documentation	97
4.11.2.1 PlaylistPrivacy	97
4.11.3 Function Documentation	98
4.11.3.1 operator<<()	98
4.12 playlist.h	98

4.13 include/song.h File Reference	100
4.13.1 Detailed Description	101
4.13.2 Enumeration Type Documentation	101
4.13.2.1 Genre	101
4.13.3 Function Documentation	101
4.13.3.1 operator<<()	101
4.14 song.h	102
4.15 include/songlist.h File Reference	103
4.15.1 Detailed Description	104
4.15.2 Variable Documentation	104
4.15.2.1 BLOCK_MAX_SIZE	104
4.16 songlist.h	105
4.17 include/spotify.h File Reference	105
4.17.1 Detailed Description	106
4.18 spotify.h	107
4.19 include/user.h File Reference	107
4.19.1 Detailed Description	109
4.19.2 Enumeration Type Documentation	109
4.19.2.1 Gender	109
4.19.3 Function Documentation	109
4.19.3.1 operator<<()	109
4.20 user.h	110
4.21 include/userlist.h File Reference	111
4.21.1 Detailed Description	113
4.21.2 Variable Documentation	113
4.21.2.1 MAX_PLAYLISTS	113
4.21.2.2 MAX_USERS	113
4.22 userlist.h	113
4.23 src/album.cpp File Reference	114
4.23.1 Detailed Description	115
4.23.2 Function Documentation	115
4.23.2.1 operator<<()	115
4.24 src/albumlist.cpp File Reference	115
4.24.1 Detailed Description	116
4.25 src/artist.cpp File Reference	116
4.25.1 Detailed Description	117
4.26 src/artistlist.cpp File Reference	117
4.26.1 Detailed Description	118
4.27 src/date.cpp File Reference	118
4.27.1 Detailed Description	119
4.28 src/main.cpp File Reference	119
4.28.1 Function Documentation	120

4.28.1.1 main()	120
4.29 src/playlist.cpp File Reference	121
4.29.1 Detailed Description	122
4.29.2 Function Documentation	122
4.29.2.1 operator<<()	122
4.30 src/song.cpp File Reference	123
4.30.1 Detailed Description	124
4.31 src/songlist.cpp File Reference	124
4.31.1 Detailed Description	125
4.32 src/spotifyfy.cpp File Reference	125
4.32.1 Detailed Description	126
4.33 src/user.cpp File Reference	126
4.34 src/userlist.cpp File Reference	127
4.34.1 Detailed Description	128
Index	129

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Album	Representa un álbum musical, con su título, artista, fecha de lanzamiento y canciones	5
AlbumList	Representa una lista dinámica de álbumes	13
Artist	Representa un artista con su nombre	19
ArtistList	Representa una lista dinámica de artistas	21
Block	Estructura que representa un bloque de canciones en la lista de canciones	23
Date	Representa una fecha con día, mes y año	25
Playlist	Representa una lista de reproducción creada por un usuario, con un título, una fecha de creación, una lista de canciones y una configuración de privacidad	32
Song	Representa una canción con un título, un género y una duración	41
SongList	Representa una lista dinámica de canciones utilizando bloques	46
Spotify	Representa la aplicación principal de gestión de música, que contiene listas de artistas, álbumes, canciones y usuarios	51
User	Representa a un usuario	64
UserList	Representa una lista de usuarios	76

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/album.h	
Declaración de la clase Album	83
include/albumlist.h	
Declaración de la clase AlbumList	86
include/artist.h	
Declaración de la clase Artist	89
include/artistlist.h	
Declaración de la clase ArtistList	91
include/date.h	
Declaración de la clase Date	93
include/playlist.h	
Declaración de la clase Playlist	96
include/song.h	
Declaración de la clase Song	100
include/songlist.h	
Declaración de la clase SongList	103
include/spotify.h	
Declaración de la clase Spotify	105
include/user.h	
Declaración de la clase User	107
include/userlist.h	
Declaración de la clase UserList	111
src/album.cpp	
Implementación de la clase Album	114
src/albumlist.cpp	
Implementación de la clase AlbumList	115
src/artist.cpp	
Implementación de la clase Artist	116
src/artistlist.cpp	
Implementación de la clase ArtistList	117
src/date.cpp	
Implementación de la clase Date	118
src/main.cpp	119
src/playlist.cpp	
Implementación de la clase Playlist	121

src/ song.cpp	
Implementación de la clase Song	123
src/ songlist.cpp	
Implementación de la clase SongList	124
src/ spotify.cpp	
Implementación de la clase Spotify	125
src/ user.cpp	126
src/ userlist.cpp	
Implementación de la clase UserList	127

Chapter 3

Class Documentation

3.1 Album Class Reference

Representa un álbum musical, con su título, artista, fecha de lanzamiento y canciones.

```
#include <album.h>
```

Collaboration diagram for Album:

Album
+ Album() + Album(string title, Artist *artist, Date releaseDate, Song **songs, int num_songs) + Album(const Album &album) + ~Album() + string getTitle() const + Artist * getArtist () const + Date getReleaseDate () const + int getNumSongs() const + Song * getSng(int i) const + void setTitle(string title) + void setArtist(Artist *artist) + void setReleaseDate (Date releaseDate) + void setNewSongs(int num_songs, Song **songs) + Album & operator=(const Album &album) + const Song & operator [](int i) const + bool addSong(Song *song)

Public Member Functions

- [Album](#) ()
Constructor por defecto.
- [Album](#) (string title, [Artist](#) *artist, [Date](#) releaseDate, [Song](#) **songs, int num_songs)
Constructor con parámetros.
- [Album](#) (const [Album](#) &album)
Constructor copia.
- [~Album](#) ()
Destructor.
- string [getTitle](#) () const

- Obtiene el título del álbum.*
- `Artist * getArtist () const`
Obtiene el artista del álbum.
- `Date getReleaseDate () const`
Obtiene la fecha de lanzamiento del álbum.
- `int getNumSongs () const`
Obtiene el número de canciones en el álbum.
- `Song * getSng (int i) const`
Obtiene una canción del álbum.
- `void setTitle (string title)`
Establece el título del álbum.
- `void setArtist (Artist *artist)`
Establece el artista del álbum.
- `void setReleaseDate (Date releaseDate)`
Establece la fecha de lanzamiento del álbum.
- `void setNewSongs (int num_songs, Song **songs)`
Establece las canciones del álbum.
- `Album & operator= (const Album &album)`
Sobrecarga del operador de asignación.
- `const Song & operator[] (int i) const`
Sobrecarga del operador de indexación.
- `bool addSong (Song *song)`
Añade una canción al álbum.

3.1.1 Detailed Description

Representa un álbum musical, con su título, artista, fecha de lanzamiento y canciones.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Album() [1/3]

```
Album::Album ( )
```

Constructor por defecto.

3.1.2.2 Album() [2/3]

```
Album::Album (
    string title,
    Artist * artist,
    Date releaseDate,
    Song ** songs,
    int num_songs )
```

Constructor con parámetros.

Parameters

<i>title</i>	Título del álbum.
<i>artist</i>	Puntero al artista del álbum.
<i>releaseDate</i>	Fecha de lanzamiento del álbum.
<i>songs</i>	Array de punteros a las canciones del álbum.
<i>num_songs</i>	Número de canciones en el álbum.

3.1.2.3 Album() [3/3]

```
Album::Album (
    const Album & album )    [inline]
```

Constructor copia.

Parameters

<i>album</i>	El álbum a copiar.
--------------	--------------------

3.1.2.4 ~Album()

```
Album::~~Album ( )
```

Destructor.

Warning

Sólo libera la memoria del array de punteros a canciones, no de las canciones ni del artista, ya que el álbum no es el propietario de esos objetos.

3.1.3 Member Function Documentation**3.1.3.1 addSong()**

```
bool Album::addSong (
    Song * song )
```

Añade una canción al álbum.

Parameters

<i>song</i>	La canción a añadir.
-------------	----------------------

Returns

true si la canción se añadió correctamente, false en caso contrario.

Parameters

<i>song</i>	Puntero a la canción a añadir.
-------------	--------------------------------

Returns

true si la canción se añadió correctamente, false en caso contrario.

Here is the caller graph for this function:

3.1.3.2 `getArtist()`

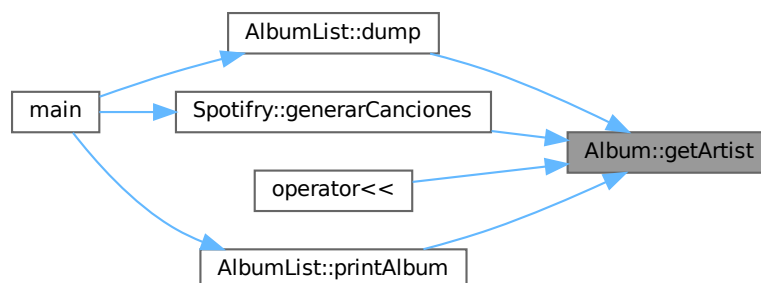
```
Artist * Album::getArtist ( ) const [inline]
```

Obtiene el artista del álbum.

Returns

Puntero al artista del álbum.

Here is the caller graph for this function:



3.1.3.3 getNumSongs()

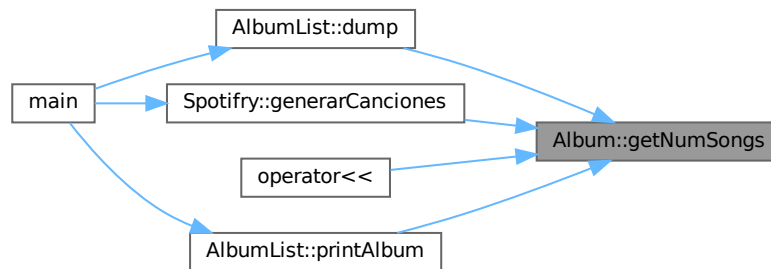
```
int Album::getNumSongs ( ) const [inline]
```

Obtiene el número de canciones en el álbum.

Returns

El número de canciones en el álbum.

Here is the caller graph for this function:



3.1.3.4 getReleaseDate()

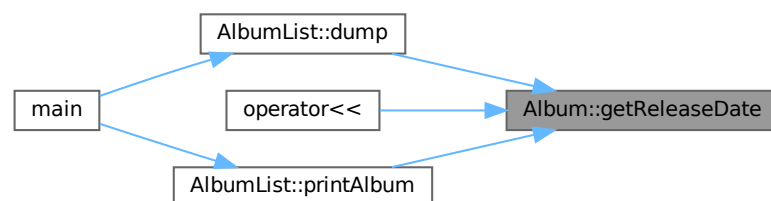
```
Date Album::getReleaseDate ( ) const [inline]
```

Obtiene la fecha de lanzamiento del álbum.

Returns

La fecha de lanzamiento del álbum.

Here is the caller graph for this function:



3.1.3.5 getSong()

```
Song * Album::getSong (
    int i ) const [inline]
```

Obtiene una canción del álbum.

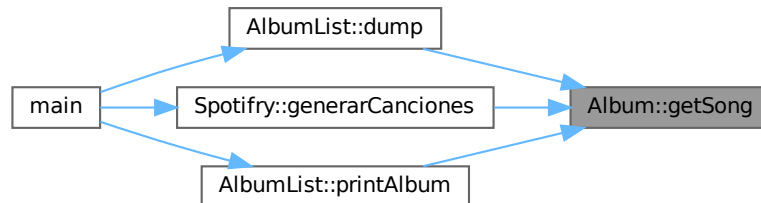
Parameters

<i>i</i>	Índice de la canción a obtener.
----------	---------------------------------

Returns

Puntero a la canción solicitada.

Here is the caller graph for this function:



3.1.3.6 getTitle()

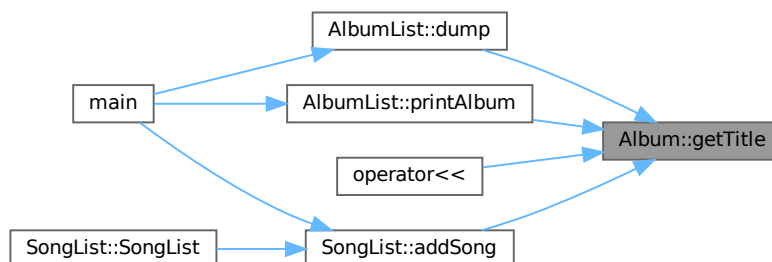
```
string Album::getTitle ( ) const [inline]
```

Obtiene el título del álbum.

Returns

El título del álbum.

Here is the caller graph for this function:



3.1.3.7 operator=()

```
Album & Album::operator= (
    const Album & album )
```

Sobrecarga del operador de asignación.

Parameters

<i>album</i>	El álbum a asignar.
--------------	---------------------

Returns

Referencia al álbum asignado.

3.1.3.8 operator[]()

```
const Song & Album::operator[] (
    int i ) const [inline]
```

Sobrecarga del operador de indexación.

Parameters

<i>i</i>	Índice de la canción a obtener.
----------	---------------------------------

Returns

Referencia constante a la canción solicitada.

3.1.3.9 setArtist()

```
void Album::setArtist (
    Artist * artist ) [inline]
```

Establece el artista del álbum.

Parameters

<i>artist</i>	El nuevo artista del álbum.
---------------	-----------------------------

3.1.3.10 setNewSongs()

```
void Album::setNewSongs (
    int num_songs,
    Song ** songs )
```

Establece las canciones del álbum.

Parameters

<i>num_songs</i>	El número de canciones en el álbum.
<i>songs</i>	El array de punteros a las canciones del álbum.

3.1.3.11 setReleaseDate()

```
void Album::setReleaseDate (
    Date releaseDate ) [inline]
```

Establece la fecha de lanzamiento del álbum.

Parameters

<i>releaseDate</i>	La nueva fecha de lanzamiento del álbum.
--------------------	--

3.1.3.12 setTitle()

```
void Album::setTitle (
    string title ) [inline]
```

Establece el título del álbum.

Parameters

<i>title</i>	El nuevo título del álbum.
--------------	----------------------------

The documentation for this class was generated from the following files:

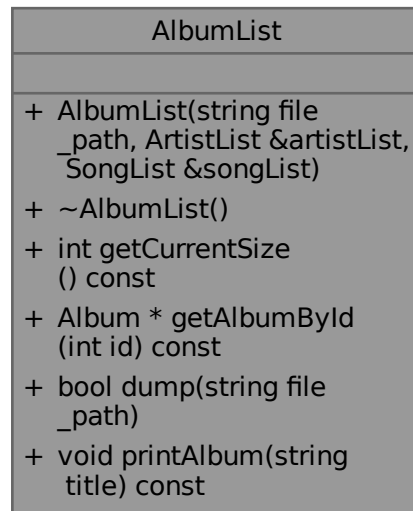
- include/[album.h](#)
- src/[album.cpp](#)

3.2 AlbumList Class Reference

Representa una lista dinámica de álbumes.

```
#include <albumlist.h>
```

Collaboration diagram for AlbumList:



Public Member Functions

- [AlbumList](#) (string file_path, [ArtistList](#) &artistList, [SongList](#) &songList)
Constructor con parámetros.
- [~AlbumList](#) ()
Destructor.
- int [getCurrentSize](#) () const
Obtiene el número actual de álbumes en la lista.
- [Album](#) * [getAlbumById](#) (int id) const
Obtiene un álbum por su posición en la lista.
- bool [dump](#) (string file_path)
Guarda la lista de álbumes en un archivo .txt.
- void [printAlbum](#) (string title) const
Imprime la información de un álbum dado su título.

3.2.1 Detailed Description

Representa una lista dinámica de álbumes.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 AlbumList()

```

AlbumList::AlbumList (
    string file_path,
    ArtistList & artistList,
    SongList & songList )

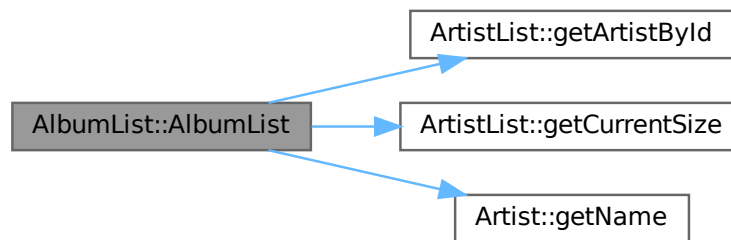
```

Constructor con parámetros.

Parameters

<i>file_path</i>	Ruta del archivo .txt con los datos de los álbumes.
<i>artistList</i>	Referencia a la lista de artistas para asociar los artistas a los álbumes.
<i>songList</i>	Referencia a la lista de canciones para asociar las canciones a los álbumes.

Here is the call graph for this function:



3.2.2.2 ~AlbumList()

```
AlbumList::~~AlbumList ( )
```

Destructor.

3.2.3 Member Function Documentation

3.2.3.1 dump()

```
bool AlbumList::dump (
    string file_path )
```

Guarda la lista de álbumes en un archivo .txt.

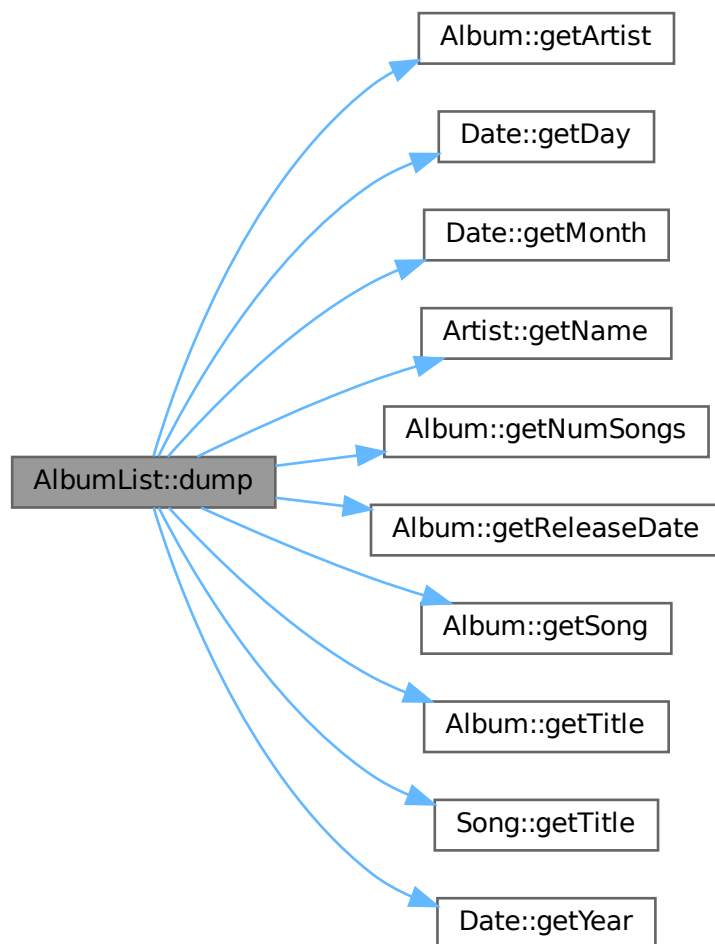
Parameters

<i>file_path</i>	Ruta del archivo .txt donde se guardarán los datos de los álbumes.
------------------	--

Returns

true si la operación fue exitosa, false en caso contrario.

Here is the call graph for this function:



Here is the caller graph for this function:



3.2.3.2 getAlbumById()

```
Album * AlbumList::getAlbumById (
    int id ) const [inline]
```

Obtiene un álbum por su posición en la lista.

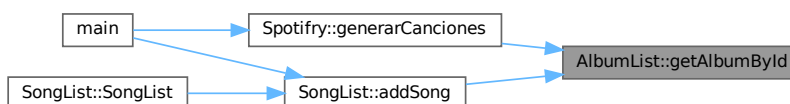
Parameters

<i>id</i>	Índice del álbum a obtener.
-----------	-----------------------------

Returns

Puntero al álbum solicitado o nullptr si no se encuentra.

Here is the caller graph for this function:



3.2.3.3 getCurrentSize()

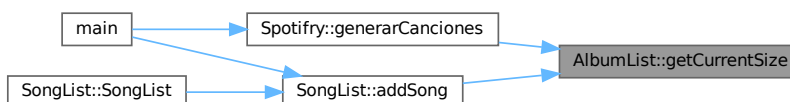
```
int AlbumList::getCurrentSize ( ) const [inline]
```

Obtiene el número actual de álbumes en la lista.

Returns

Número de álbumes en la lista.

Here is the caller graph for this function:



3.2.3.4 printAlbum()

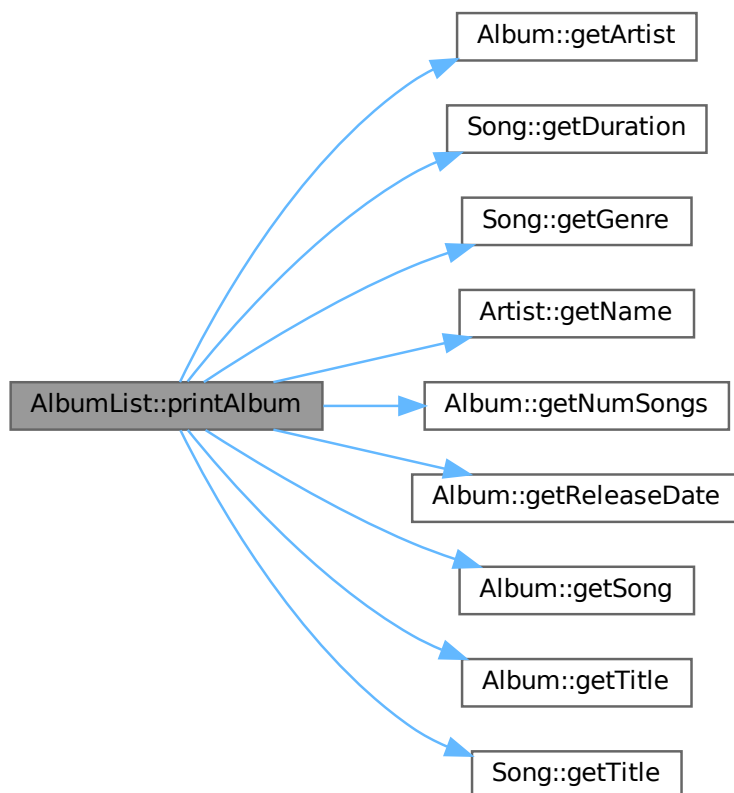
```
void AlbumList::printAlbum (
    string title ) const
```

Imprime la información de un álbum dado su título.

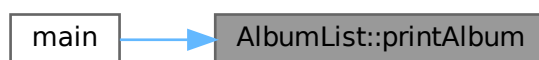
Parameters

<i>title</i>	El título del álbum a imprimir.
--------------	---------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

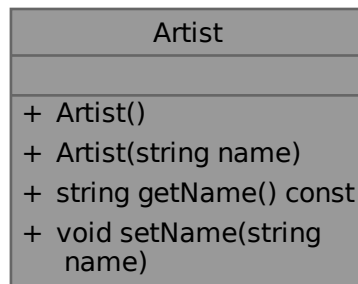
- [include/albumlist.h](#)
- [src/albumlist.cpp](#)

3.3 Artist Class Reference

Representa un artista con su nombre.

```
#include <artist.h>
```

Collaboration diagram for Artist:



Public Member Functions

- [Artist](#) ()
Constructor por defecto.
- [Artist](#) (string name)
Constructor con parámetros.
- string [getName](#) () const
Obtiene el nombre del artista.
- void [setName](#) (string name)
Establece el nombre del artista.

3.3.1 Detailed Description

Representa un artista con su nombre.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 [Artist\(\)](#) [1/2]

```
Artist::Artist ( ) [inline]
```

Constructor por defecto.

3.3.2.2 [Artist\(\)](#) [2/2]

```
Artist::Artist (  
    string name ) [inline]
```

Constructor con parámetros.

Parameters

<i>name</i>	Nombre del artista.
-------------	---------------------

3.3.3 Member Function Documentation

3.3.3.1 getName()

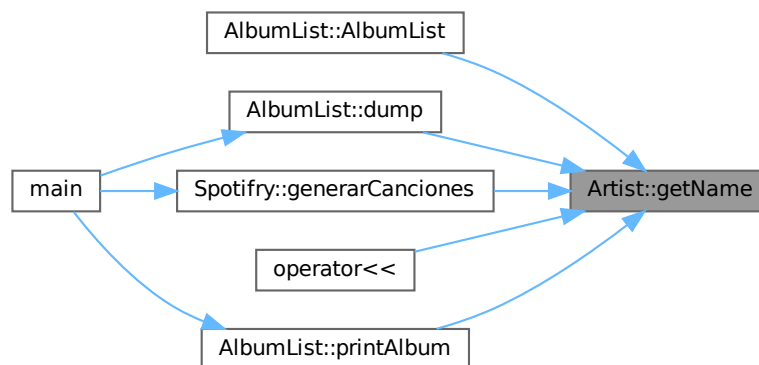
```
string Artist::getName ( ) const [inline]
```

Obtiene el nombre del artista.

Returns

Nombre del artista.

Here is the caller graph for this function:



3.3.3.2 setName()

```
void Artist::setName (
    string name ) [inline]
```

Establece el nombre del artista.

Parameters

<i>name</i>	Nuevo nombre del artista.
-------------	---------------------------

The documentation for this class was generated from the following file:

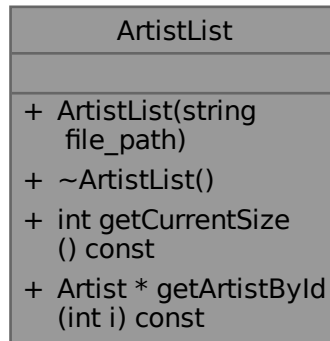
- include/[artist.h](#)

3.4 ArtistList Class Reference

Representa una lista dinámica de artistas.

```
#include <artistlist.h>
```

Collaboration diagram for ArtistList:



Public Member Functions

- [ArtistList](#) (string file_path)
Constructor con parámetros.
- [~ArtistList](#) ()
Destructor.
- int [getCurrentSize](#) () const
Obtiene el número actual de artistas en la lista.
- [Artist *](#) [getArtistById](#) (int i) const
Obtiene un artista por su posición en la lista.

3.4.1 Detailed Description

Representa una lista dinámica de artistas.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 ArtistList()

```
ArtistList::ArtistList (  
    string file_path )
```

Constructor con parámetros.

Parameters

<i>file_path</i>	Ruta del archivo .txt con los datos de los artistas.
------------------	--

3.4.2.2 ~ArtistList()

```
ArtistList::~~ArtistList ( )
```

Destructor.

3.4.3 Member Function Documentation**3.4.3.1 getArtistById()**

```
Artist * ArtistList::getArtistById (
    int i ) const [inline]
```

Obtiene un artista por su posición en la lista.

Parameters

<i>id</i>	Índice del artista a obtener.
-----------	-------------------------------

Returns

Puntero al artista solicitado o nullptr si no se encuentra.

Here is the caller graph for this function:

**3.4.3.2 getCurrentSize()**

```
int ArtistList::getCurrentSize ( ) const [inline]
```

Obtiene el número actual de artistas en la lista.

Returns

Número de artistas en la lista.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

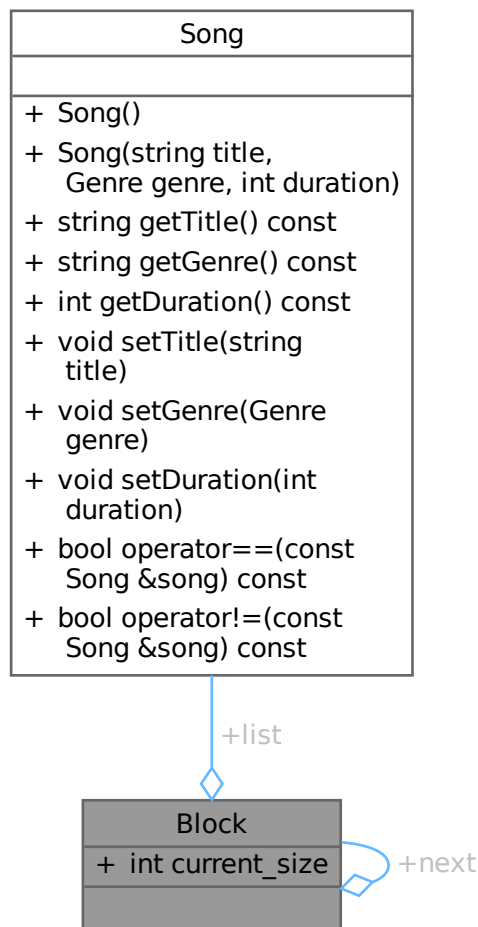
- [include/artistlist.h](#)
- [src/artistlist.cpp](#)

3.5 Block Struct Reference

Estructura que representa un bloque de canciones en la lista de canciones.

```
#include <songlist.h>
```

Collaboration diagram for Block:



Public Attributes

- `Song * list [BLOCK_MAX_SIZE]`
Array de punteros a canciones en el bloque.
- `int current_size`
Número actual de canciones en el bloque.
- `Block * next`
Puntero al siguiente bloque de canciones.

3.5.1 Detailed Description

Estructura que representa un bloque de canciones en la lista de canciones.

3.5.2 Member Data Documentation

3.5.2.1 `current_size`

```
int Block::current_size
```

Número actual de canciones en el bloque.

3.5.2.2 `list`

```
Song* Block::list[BLOCK_MAX_SIZE]
```

Array de punteros a canciones en el bloque.

3.5.2.3 `next`

```
Block* Block::next
```

Puntero al siguiente bloque de canciones.

The documentation for this struct was generated from the following file:

- `include/songlist.h`

3.6 Date Class Reference

Representa una fecha con día, mes y año.

```
#include <date.h>
```

Collaboration diagram for Date:

Date
+ Date() + Date(int day, int month, int year) + int getDay() const + int getMonth() const + int getYear() const + void setDay(int day) + void setMonth(int month) + void setYear(int year) + bool operator==(const Date &date) const + bool operator!=(const Date &date) const + bool operator<(const Date &date) const + bool operator>(const Date &date) const + bool operator>=(const Date &date) const + bool operator<=(const Date &date) const

Public Member Functions

- [Date](#) ()
Constructor por defecto.
- [Date](#) (int day, int month, int year)
Constructor con parámetros.
- int [getDay](#) () const
Obtiene el día.
- int [getMonth](#) () const
Obtiene el mes.
- int [getYear](#) () const
Obtiene el año.
- void [setDay](#) (int day)
Establece el día.
- void [setMonth](#) (int month)
Establece el mes.
- void [setYear](#) (int year)

Establece el año.

- bool `operator==` (const `Date` &date) const

Sobrecarga del operador de igualdad.

- bool `operator!=` (const `Date` &date) const

Sobrecarga del operador de desigualdad.

- bool `operator<` (const `Date` &date) const

Sobrecarga del operador de comparación menor que.

- bool `operator>` (const `Date` &date) const

Sobrecarga del operador de comparación mayor que.

- bool `operator>=` (const `Date` &date) const

Sobrecarga del operador de comparación mayor o igual que.

- bool `operator<=` (const `Date` &date) const

Sobrecarga del operador de comparación menor o igual que.

3.6.1 Detailed Description

Representa una fecha con día, mes y año.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 `Date()` [1/2]

```
Date::Date ( )
```

Constructor por defecto.

3.6.2.2 `Date()` [2/2]

```
Date::Date (
    int day,
    int month,
    int year )
```

Constructor con parámetros.

Parameters

<i>day</i>	Día
<i>month</i>	Mes
<i>year</i>	Año
<i>day</i>	Día (1-31)
<i>month</i>	Mes (1-12)
<i>year</i>	Año (ej. 2026)

3.6.3 Member Function Documentation

3.6.3.1 `getDay()`

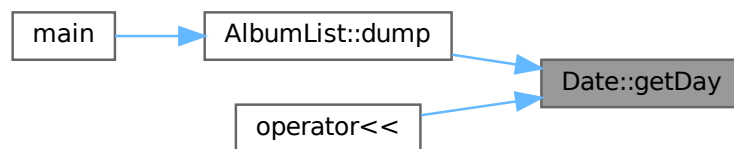
```
int Date::getDay ( ) const [inline]
```

Obtiene el día.

Returns

Día.

Here is the caller graph for this function:



3.6.3.2 `getMonth()`

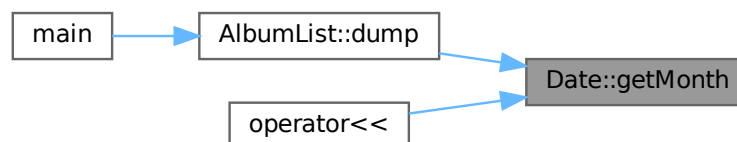
```
int Date::getMonth ( ) const [inline]
```

Obtiene el mes.

Returns

Mes.

Here is the caller graph for this function:



3.6.3.3 getYear()

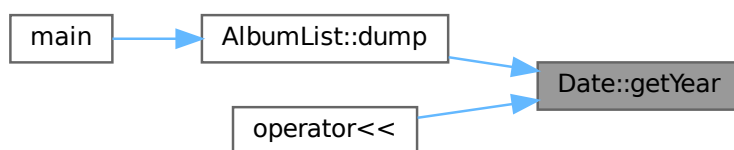
```
int Date::getYear ( ) const [inline]
```

Obtiene el año.

Returns

Año.

Here is the caller graph for this function:



3.6.3.4 operator"!="()

```
bool Date::operator!= (
    const Date & date ) const [inline]
```

Sobrecarga del operador de desigualdad.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si las fechas son diferentes, false en caso contrario.

3.6.3.5 operator<()

```
bool Date::operator< (
    const Date & date ) const
```

Sobrecarga del operador de comparación menor que.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si esta fecha es anterior a la fecha dada, false en caso contrario.

3.6.3.6 operator<=()

```
bool Date::operator<= (
    const Date & date ) const [inline]
```

Sobrecarga del operador de comparación menor o igual que.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si esta fecha es anterior o igual a la fecha dada, false en caso contrario.

3.6.3.7 operator==()

```
bool Date::operator== (
    const Date & date ) const [inline]
```

Sobrecarga del operador de igualdad.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si las fechas son iguales, false en caso contrario.

3.6.3.8 operator>()

```
bool Date::operator> (
    const Date & date ) const [inline]
```

Sobrecarga del operador de comparación mayor que.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si esta fecha es posterior a la fecha dada, false en caso contrario.

3.6.3.9 operator>=()

```
bool Date::operator>= (
    const Date & date ) const [inline]
```

Sobrecarga del operador de comparación mayor o igual que.

Parameters

<i>date</i>	Fecha a comparar.
-------------	-------------------

Returns

true si esta fecha es posterior o igual a la fecha dada, false en caso contrario.

3.6.3.10 setDay()

```
void Date::setDay (
    int day ) [inline]
```

Establece el día.

Parameters

<i>day</i>	Día.
------------	------

3.6.3.11 setMonth()

```
void Date::setMonth (
    int month ) [inline]
```

Establece el mes.

Parameters

<i>month</i>	Mes.
--------------	------

3.6.3.12 setYear()

```
void Date::setYear (
    int year ) [inline]
```

Establece el año.

Parameters

<i>year</i>	Año.
-------------	------

The documentation for this class was generated from the following files:

- [include/date.h](#)
- [src/date.cpp](#)

3.7 Playlist Class Reference

Representa una lista de reproducción creada por un usuario, con un título, una fecha de creación, una lista de canciones y una configuración de privacidad.

```
#include <playlist.h>
```

Collaboration diagram for Playlist:

Playlist
+ Playlist() + Playlist(string title, User *creator, Date creationDate, PlaylistPrivacy privacy) + Playlist(const Playlist &playlist) + ~Playlist() + string getTitle() const + const User & getCreator() const + Date getCreationDate() const + int getMaxSongs() const + int getNumSongs() const + PlaylistPrivacy getPrivacy() const + void setTitle(string title) + void setCreationDate(Date creationDate) + void setPrivacy(PlaylistPrivacy privacy) + bool addSong(Song *song, User *whoAddsIt) + bool deleteSong(Song *song, User *whoRemovesIt) + Playlist & operator=(const Playlist &playlist) + bool operator==(const Playlist &playlist) + bool operator!=(const Playlist &playlist) + const Song & operator[](int i) const

Public Member Functions

- [Playlist \(\)](#)
Constructor por defecto.
- [Playlist](#) (string title, [User](#) *creator, [Date](#) creationDate, [PlaylistPrivacy](#) privacy)
Constructor con parámetros.
- [Playlist](#) (const [Playlist](#) &playlist)

- Constructor copia.*

 - `~Playlist ()`
- Destructor.*

 - `string getTitle () const`
Obtiene el título de la playlist.
 - `const User & getCreator () const`
Obtiene el creador de la playlist.
 - `Date getCreationDate () const`
Obtiene la fecha de creación de la playlist.
 - `int getMaxSongs () const`
Obtiene el número máximo de canciones en la playlist.
 - `int getNumSongs () const`
Obtiene el número de canciones en la playlist.
 - `PlaylistPrivacy getPrivacy () const`
Obtiene la configuración de privacidad de la playlist.
 - `void setTitle (string title)`
Establece el título de la playlist.
 - `void setCreationDate (Date creationDate)`
Establece la fecha de creación de la playlist.
 - `void setPrivacy (PlaylistPrivacy privacy)`
Establece la configuración de privacidad de la playlist.
 - `bool addSong (Song *song, User *whoAddsIt)`
Añade una canción a la playlist si el usuario que la añade es el creador de la playlist.
 - `bool deleteSong (Song *song, User *whoRemovesIt)`
Elimina una canción de la playlist si el usuario que la elimina es el creador de la playlist.
 - `Playlist & operator= (const Playlist &playlist)`
Sobrecarga del operador de asignación.
 - `bool operator== (const Playlist &playlist)`
Sobrecarga del operador de igualdad.
 - `bool operator!= (const Playlist &playlist)`
Sobrecarga del operador de desigualdad.
 - `const Song & operator[] (int i) const`
Sobrecarga del operador de indexación para acceder a las canciones de la playlist.

3.7.1 Detailed Description

Representa una lista de reproducción creada por un usuario, con un título, una fecha de creación, una lista de canciones y una configuración de privacidad.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Playlist() [1/3]

```
Playlist::Playlist ( )
```

Constructor por defecto.

3.7.2.2 Playlist() [2/3]

```
Playlist::Playlist (
    string title,
    User * creator,
    Date creationDate,
    PlaylistPrivacy privacy )
```

Constructor con parámetros.

Parameters

<i>title</i>	Título de la playlist.
<i>creator</i>	Puntero al usuario creador de la playlist.
<i>creationDate</i>	Fecha de creación de la playlist.
<i>privacy</i>	Configuración de privacidad de la playlist.
<i>title</i>	El título de la playlist.
<i>creator</i>	El usuario creador de la playlist.
<i>creationDate</i>	La fecha de creación de la playlist.
<i>privacy</i>	La configuración de privacidad de la playlist.

3.7.2.3 Playlist() [3/3]

```
Playlist::Playlist (
    const Playlist & playlist ) [inline]
```

Constructor copia.

Parameters

<i>playlist</i>	La playlist a copiar.
-----------------	-----------------------

3.7.2.4 ~Playlist()

```
Playlist::~~Playlist ( )
```

Destructor.

Warning

Sólo libera la memoria del array de punteros a canciones, no de las canciones ni del creador, ya que la playlist no es el propietario de esos objetos.

3.7.3 Member Function Documentation

3.7.3.1 addSong()

```
bool Playlist::addSong (
    Song * song,
    User * whoAddsIt )
```

Añade una canción a la playlist si el usuario que la añade es el creador de la playlist.

Añade una canción a la playlist si el usuario que la agrega es el creador de la playlist.

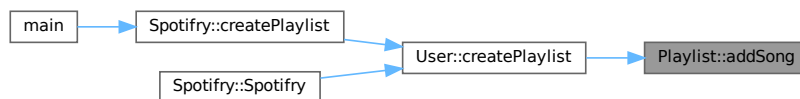
Parameters

<i>song</i>	La canción a añadir.
<i>who</i> ↔ <i>AddsIt</i>	El usuario que intenta añadir la canción.

Returns

true si la canción fue añadida exitosamente, false en caso contrario.

Here is the caller graph for this function:



3.7.3.2 deleteSong()

```

bool Playlist::deleteSong (
    Song * song,
    User * whoRemovesIt )
  
```

Elimina una canción de la playlist si el usuario que la elimina es el creador de la playlist.

Parameters

<i>song</i>	La canción a eliminar.
<i>who</i> ↔ <i>RemovesIt</i>	El usuario que intenta eliminar la canción.

Returns

true si la canción fue eliminada exitosamente, false en caso contrario.

3.7.3.3 getCreationDate()

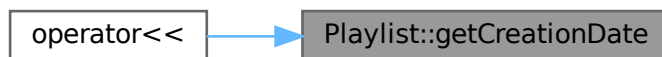
```
Date Playlist::getCreationDate ( ) const [inline]
```

Obtiene la fecha de creación de la playlist.

Returns

La fecha de creación de la playlist.

Here is the caller graph for this function:

**3.7.3.4 getCreator()**

```
const User & Playlist::getCreator ( ) const [inline]
```

Obtiene el creador de la playlist.

Returns

Una referencia constante al usuario creador de la playlist.

Here is the caller graph for this function:

**3.7.3.5 getMaxSongs()**

```
int Playlist::getMaxSongs ( ) const [inline]
```

Obtiene el número máximo de canciones en la playlist.

Returns

El número máximo de canciones en la playlist.

3.7.3.6 getNumSongs()

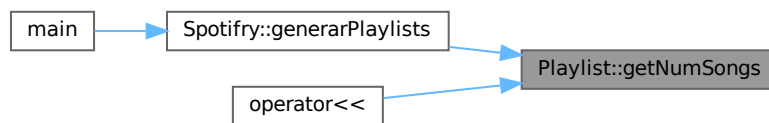
```
int Playlist::getNumSongs ( ) const [inline]
```

Obtiene el número de canciones en la playlist.

Returns

El número de canciones en la playlist.

Here is the caller graph for this function:



3.7.3.7 getPrivacy()

```
PlaylistPrivacy Playlist::getPrivacy ( ) const [inline]
```

Obtiene la configuración de privacidad de la playlist.

Returns

La configuración de privacidad de la playlist.

3.7.3.8 getTitle()

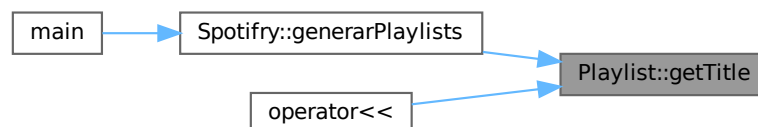
```
string Playlist::getTitle ( ) const [inline]
```

Obtiene el título de la playlist.

Returns

El título de la playlist.

Here is the caller graph for this function:



3.7.3.9 operator!=(())

```
bool Playlist::operator!= (
    const Playlist & playlist ) [inline]
```

Sobrecarga del operador de desigualdad.

Parameters

<i>playlist</i>	La playlist a comparar con esta.
-----------------	----------------------------------

Returns

true si las playlists son diferentes, false en caso contrario.

3.7.3.10 operator=()

```
Playlist & Playlist::operator= (
    const Playlist & playlist )
```

Sobrecarga del operador de asignación.

Parameters

<i>playlist</i>	La playlist a asignar a esta.
-----------------	-------------------------------

Returns

Una referencia a esta playlist después de la asignación.

3.7.3.11 operator==(())

```
bool Playlist::operator== (
    const Playlist & playlist ) [inline]
```

Sobrecarga del operador de igualdad.

Parameters

<i>playlist</i>	La playlist a comparar con esta.
-----------------	----------------------------------

Returns

true si las playlists son iguales (título y creador), false en caso contrario.

3.7.3.12 operator[]()

```
const Song & Playlist::operator[] (
```

```
int i ) const [inline]
```

Sobrecarga del operador de indexación para acceder a las canciones de la playlist.

Parameters

<i>i</i>	El índice de la canción a acceder.
----------	------------------------------------

Returns

Una referencia constante a la canción en la posición *i* de la playlist.

3.7.3.13 setCreationDate()

```
void Playlist::setCreationDate (
    Date creationDate ) [inline]
```

Establece la fecha de creación de la playlist.

Parameters

<i>creationDate</i>	La nueva fecha de creación de la playlist.
---------------------	--

Here is the caller graph for this function:



3.7.3.14 setPrivacy()

```
void Playlist::setPrivacy (
    PlaylistPrivacy privacy ) [inline]
```

Establece la configuración de privacidad de la playlist.

Parameters

<i>privacy</i>	La nueva configuración de privacidad de la playlist.
----------------	--

Here is the caller graph for this function:



3.7.3.15 setTitle()

```
void Playlist::setTitle (
    string title ) [inline]
```

Establece el título de la playlist.

Parameters

<i>title</i>	El nuevo título de la playlist.
--------------	---------------------------------

The documentation for this class was generated from the following files:

- [include/playlist.h](#)
- [src/playlist.cpp](#)

3.8 Song Class Reference

Representa una canción con un título, un género y una duración.

```
#include <song.h>
```

Collaboration diagram for Song:

Song
+ Song() + Song(string title, Genre genre, int duration) + string getTitle() const + string getGenre() const + int getDuration() const + void setTitle(string title) + void setGenre(Genre genre) + void setDuration(int duration) + bool operator==(const Song &song) const + bool operator!=(const Song &song) const

Public Member Functions

- [Song](#) ()
Constructor por defecto.
- [Song](#) (string title, [Genre](#) genre, int duration)
Constructor con parámetros.
- string [getTitle](#) () const
Obtiene el título de la canción.
- string [getGenre](#) () const
Obtiene el género de la canción como una cadena de texto.
- int [getDuration](#) () const
Obtiene la duración de la canción en segundos.
- void [setTitle](#) (string title)
Establece el título de la canción.
- void [setGenre](#) ([Genre](#) genre)
Establece el género de la canción.
- void [setDuration](#) (int duration)
Establece la duración de la canción.
- bool [operator==](#) (const [Song](#) &song) const
Sobrecarga del operador de igualdad.
- bool [operator!=](#) (const [Song](#) &song) const
Sobrecarga del operador de desigualdad para comparar dos canciones.

3.8.1 Detailed Description

Representa una canción con un título, un género y una duración.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 Song() [1/2]

```
Song::Song ( )
```

Constructor por defecto.

3.8.2.2 Song() [2/2]

```
Song::Song (
    string title,
    Genre genre,
    int duration )
```

Constructor con parámetros.

Parameters

<i>title</i>	El título de la canción.
<i>genre</i>	El género de la canción.
<i>duration</i>	La duración de la canción en segundos.

3.8.3 Member Function Documentation

3.8.3.1 getDuration()

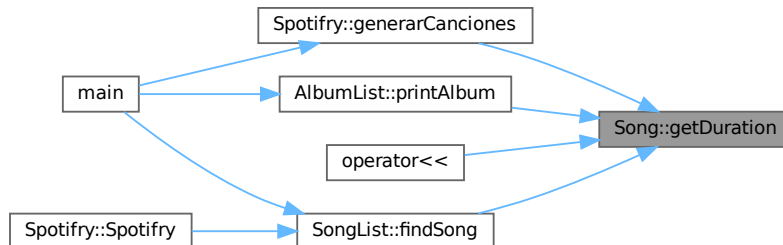
```
int Song::getDuration ( ) const [inline]
```

Obtiene la duración de la canción en segundos.

Returns

La duración de la canción en segundos.

Here is the caller graph for this function:

**3.8.3.2 getGenre()**

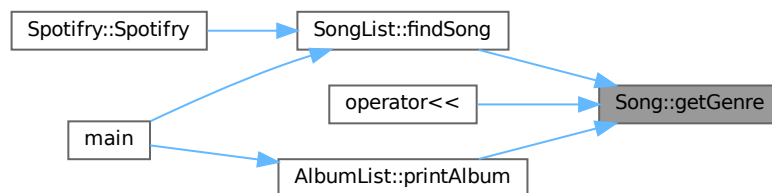
```
string Song::getGenre ( ) const
```

Obtiene el género de la canción como una cadena de texto.

Returns

El género de la canción como una cadena de texto.

Here is the caller graph for this function:

**3.8.3.3 getTitle()**

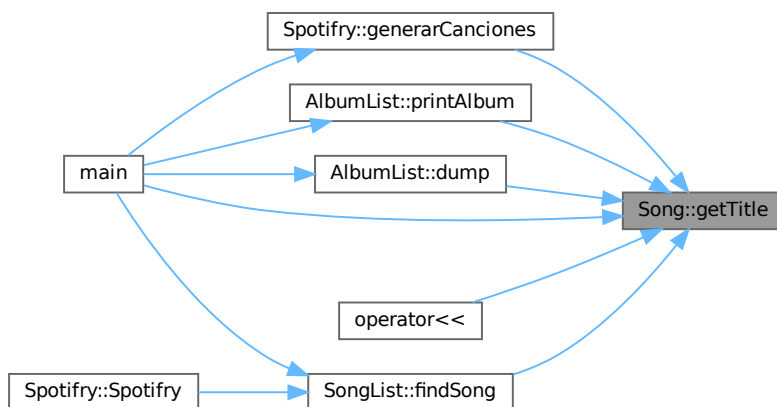
```
string Song::getTitle ( ) const [inline]
```

Obtiene el título de la canción.

Returns

El título de la canción.

Here is the caller graph for this function:

**3.8.3.4 operator"!=()**

```
bool Song::operator!= (
    const Song & song ) const [inline]
```

Sobrecarga del operador de desigualdad para comparar dos canciones.

Parameters

<i>song</i>	La canción a comparar con esta.
-------------	---------------------------------

Returns

true si las canciones son diferentes, false en caso contrario.

3.8.3.5 operator==(

```
bool Song::operator== (
    const Song & song ) const [inline]
```

Sobrecarga del operador de igualdad.

Parameters

<i>song</i>	La canción a comparar con esta.
-------------	---------------------------------

Returns

true si las canciones son iguales (título, género y duración), false en caso contrario.

3.8.3.6 setDuration()

```
void Song::setDuration (
    int duration ) [inline]
```

Establece la duración de la canción.

Parameters

<i>duration</i>	La nueva duración de la canción en segundos.
-----------------	--

3.8.3.7 setGenre()

```
void Song::setGenre (
    Genre genre ) [inline]
```

Establece el género de la canción.

Parameters

<i>genre</i>	El nuevo género de la canción.
--------------	--------------------------------

3.8.3.8 setTitle()

```
void Song::setTitle (
    string title ) [inline]
```

Establece el título de la canción.

Parameters

<i>title</i>	El nuevo título de la canción.
--------------	--------------------------------

The documentation for this class was generated from the following files:

- include/[song.h](#)
- src/[song.cpp](#)

3.9 SongList Class Reference

Representa una lista dinámica de canciones utilizando bloques.


```
#include <songlist.h>
```

Collaboration diagram for SongList:



Public Member Functions

- [SongList](#) (string file_path, [AlbumList](#) &albumList)
Constructor con parámetros.
- [~SongList](#) ()
Destructor.
- bool [addSong](#) (string title, [Genre](#) genre, int duration, [AlbumList](#) &albumList, string album)
Añade una nueva canción a la lista.
- bool [findSong](#) (string searchTerm, [Genre](#) genre, int durationMin, int durationMax, [Song](#) **results, int maxResults, int &numResults)
Busca canciones en la lista que coincidan con los criterios de búsqueda.

3.9.1 Detailed Description

Representa una lista dinámica de canciones utilizando bloques.

3.9.2 Constructor & Destructor Documentation

3.9.2.1 SongList()

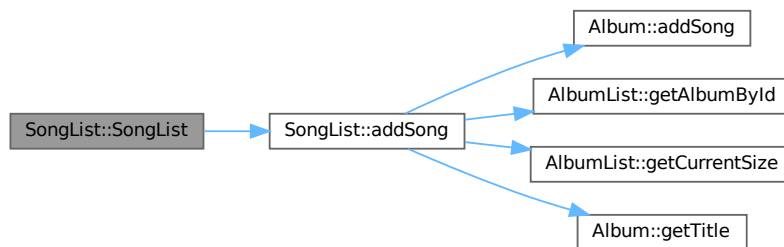
```
SongList::SongList (
    string file_path,
    AlbumList & albumList )
```

Constructor con parámetros.

Parameters

<i>file_path</i>	Ruta del archivo .txt con los datos de las canciones.
<i>albumList</i>	Referencia a la lista de álbumes para asociar las canciones a los álbumes.

Here is the call graph for this function:



3.9.2.2 ~SongList()

```
SongList::~~SongList ( )
```

Destructor.

3.9.3 Member Function Documentation

3.9.3.1 addSong()

```
bool SongList::addSong (
    string title,
    Genre genre,
    int duration,
    AlbumList & albumList,
    string album )
```

Añade una nueva canción a la lista.

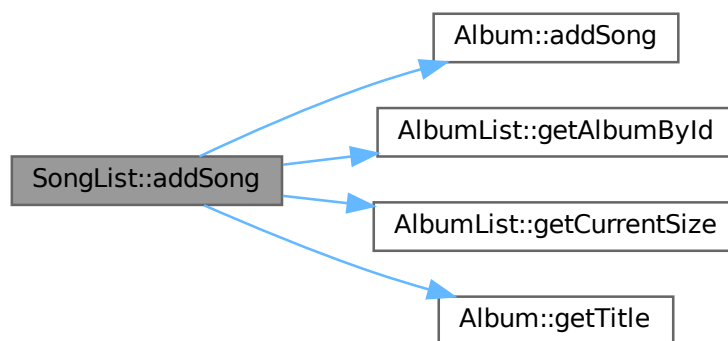
Parameters

<i>title</i>	El título de la canción.
<i>genre</i>	El género de la canción.
<i>duration</i>	La duración de la canción en segundos.
<i>albumList</i>	Referencia a la lista de álbumes para asociar la canción al álbum correspondiente.
<i>album</i>	El título del álbum al que pertenece la canción.

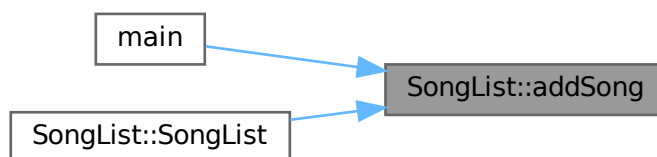
Returns

true si la canción se añadió correctamente, false en caso contrario.

Here is the call graph for this function:



Here is the caller graph for this function:



3.9.3.2 findSong()

```
bool SongList::findSong (
    string searchTerm,
    Genre genre,
    int durationMin,
    int durationMax,
    Song ** results,
    int maxResults,
    int & numResults )
```

Busca canciones en la lista que coincidan con los criterios de búsqueda.

Parameters

<i>searchTerm</i>	El término subcadena de búsqueda para el título de la canción.
<i>genre</i>	El género de la canción a buscar (puede ser UNKNOWN para ignorar este filtro).
<i>durationMin</i>	La duración mínima de la canción en segundos (puede ser -1 para ignorar este filtro).
<i>durationMax</i>	La duración máxima de la canción en segundos (puede ser -1 para ignorar este filtro).
<i>results</i>	Array de punteros a canciones donde se almacenarán los resultados encontrados.
<i>maxResults</i>	El tamaño máximo del array de resultados.
<i>numResults</i>	Variable donde se almacenará el número de resultados encontrados.

Returns

true si se encontraron canciones que coincidan con los criterios, false en caso contrario.

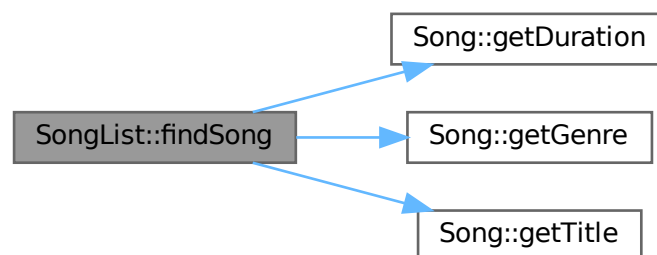
Parameters

<i>searchTerm</i>	El término de búsqueda para el título de la canción (puede ser una subcadena).
<i>genre</i>	El género de la canción a buscar (puede ser UNKNOWN para no filtrar por género).
<i>durationMin</i>	La duración mínima de la canción en segundos (puede ser -1 para no filtrar por duración mínima).
<i>durationMax</i>	La duración máxima de la canción en segundos (puede ser -1 para no filtrar por duración máxima).
<i>results</i>	Array de punteros a canciones donde se almacenarán los resultados encontrados.
<i>maxResults</i>	El tamaño máximo del array de resultados.
<i>numResults</i>	Variable donde se almacenará el número de resultados encontrados.

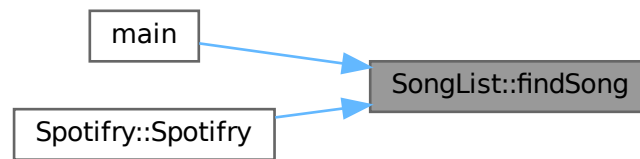
Returns

true si se encontraron canciones que coincidan con los criterios, false en caso contrario.

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

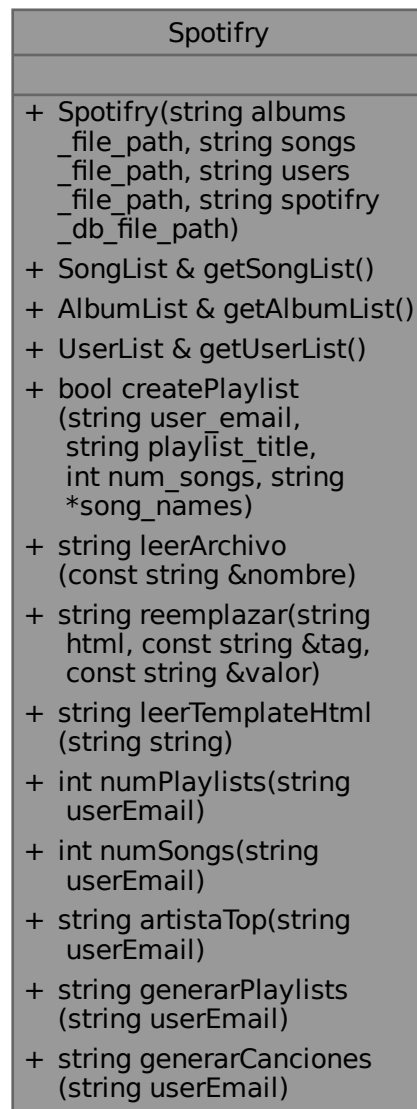
- [include/songlist.h](#)
- [src/songlist.cpp](#)

3.10 Spotifry Class Reference

Representa la aplicación principal de gestión de música, que contiene listas de artistas, álbumes, canciones y usuarios.

```
#include <spotifry.h>
```

Collaboration diagram for Spotifry:



Public Member Functions

- [Spotifry](#) (string albums_file_path, string songs_file_path, string users_file_path, string spotifry_db_file_path)
Constructor con parámetros.
- [SongList](#) & [getSongList](#) ()
Obtiene la lista de canciones.
- [AlbumList](#) & [getAlbumList](#) ()
Obtiene la lista de artistas.
- [UserList](#) & [getUserList](#) ()
Obtiene la lista de usuarios.
- bool [createPlaylist](#) (string user_email, string playlist_title, int num_songs, string *song_names)

Crea una nueva playlist para un usuario dado su email, el título de la playlist y los nombres de las canciones a incluir en la playlist.

- string `leerArchivo` (const string &nombre)

Lee el contenido de un archivo.

- string `reemplazar` (string html, const string &tag, const string &valor)

Reemplaza una etiqueta en un string HTML por un valor dado.

- string `leerTemplateHtml` (string string)

Lee una plantilla HTML desde un archivo y devuelve su contenido como un string.

- int `numPlaylists` (string userEmail)

Genera un string HTML con la información de un álbum dado su título.

- int `numSongs` (string userEmail)

Obtiene el número de canciones guardadas por un usuario dado su email.

- string `artistaTop` (string userEmail)

Obtiene un ejemplo de el artista más escuchado por un usuario dado su email.

- string `generarPlaylists` (string userEmail)

Genera un string HTML con la información de las playlists de un usuario dado su email.

- string `generarCanciones` (string userEmail)

Genera un string HTML con la información de las canciones guardadas de un usuario dado su email.

3.10.1 Detailed Description

Representa la aplicación principal de gestión de música, que contiene listas de artistas, álbumes, canciones y usuarios.

3.10.2 Constructor & Destructor Documentation

3.10.2.1 Spotify()

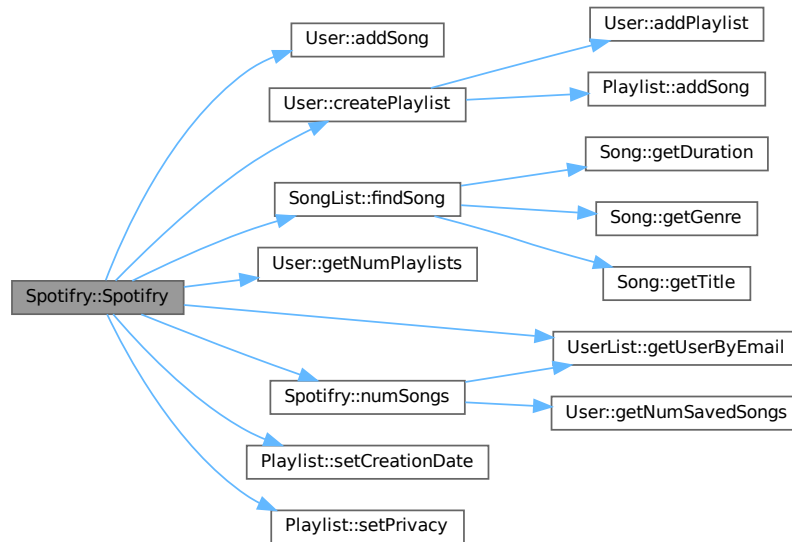
```
Spotify::Spotify (
    string albums_file_path,
    string songs_file_path,
    string users_file_path,
    string spotify_db_file_path )
```

Constructor con parámetros.

Parameters

<code>albums_file_path</code>	Ruta del archivo .txt con los datos de los álbumes.
<code>songs_file_path</code>	Ruta del archivo .txt con los datos de las canciones.
<code>users_file_path</code>	Ruta del archivo .txt con los datos de los usuarios.
<code>spotify_db_file_path</code>	Ruta del archivo .txt con los datos de las canciones y playlists de los usuarios.

Here is the call graph for this function:



3.10.3 Member Function Documentation

3.10.3.1 artistaTop()

```
string Spotify::artistaTop (
    string userEmail )
```

Obtiene un ejemplo de el artista más escuchado por un usuario dado su email.

Parameters

<i>userEmail</i>	El email del usuario para el cual se obtendrá el artista más escuchado.
------------------	---

Returns

Coldplay

Here is the call graph for this function:



Here is the caller graph for this function:



3.10.3.2 createPlaylist()

```

bool Spotify::createPlaylist (
    string user_email,
    string playlist_title,
    int num_songs,
    string * song_names )
  
```

Crea una nueva playlist para un usuario dado su email, el título de la playlist y los nombres de las canciones a incluir en la playlist.

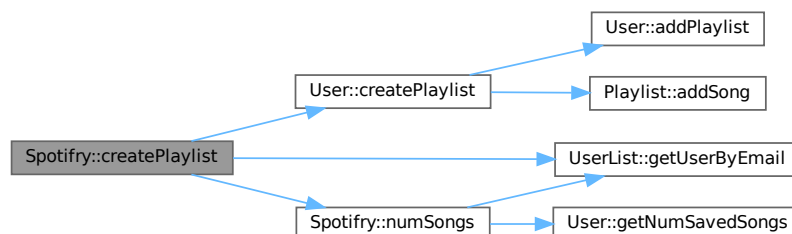
Parameters

<i>user_email</i>	El email del usuario para el cual se creará la playlist.
<i>playlist_title</i>	El título de la nueva playlist.
<i>num_songs</i>	El número de canciones a incluir en la playlist.
<i>song_names</i>	Un array de strings con los nombres de las canciones a incluir en la playlist.

Returns

true si la playlist se creó correctamente, false en caso contrario.

Here is the call graph for this function:



Here is the caller graph for this function:



3.10.3.3 generarCanciones()

```
string Spotify::generarCanciones (
    string userEmail )
```

Genera un string HTML con la información de las canciones guardadas de un usuario dado su email.

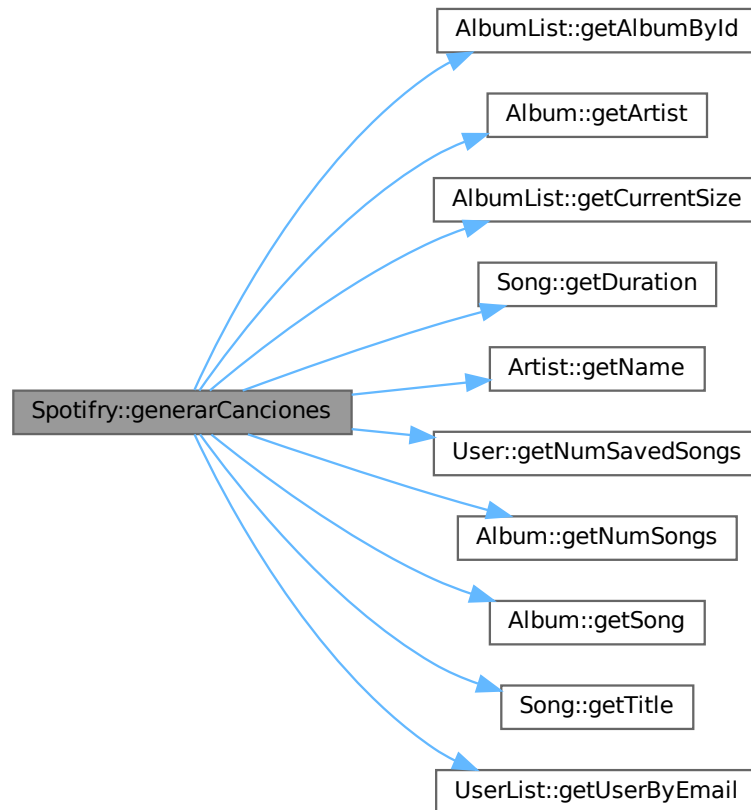
Parameters

<i>userEmail</i>	El email del usuario para el cual se generará el HTML de las canciones guardadas.
------------------	---

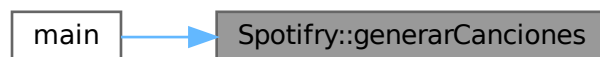
Returns

Un string HTML con la información de las canciones guardadas del usuario, o una cadena vacía si el usuario no se encuentra o no tiene canciones guardadas.

Here is the call graph for this function:



Here is the caller graph for this function:



3.10.3.4 generarPlaylists()

```
string Spotifry::generarPlaylists (
    string userEmail )
```

Genera un string HTML con la información de las playlists de un usuario dado su email.

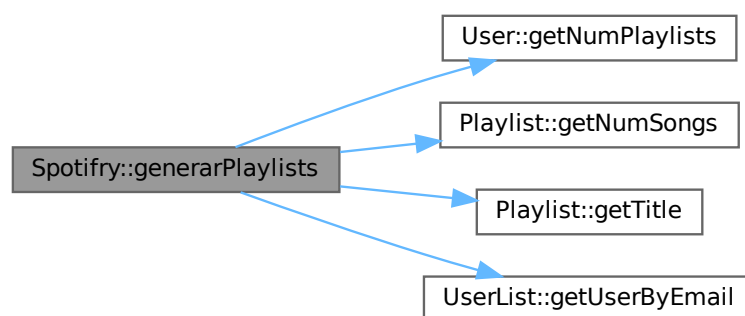
Parameters

<i>userEmail</i>	El email del usuario para el cual se generará el HTML de las playlists.
------------------	---

Returns

Un string HTML con la información de las playlists del usuario, o una cadena vacía si el usuario no se encuentra o no tiene playlists.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.10.3.5 getAlbumList()**

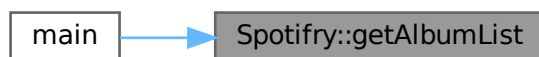
```
AlbumList & Spotifyfy::getAlbumList ( ) [inline]
```

Obtiene la lista de artistas.

Returns

Referencia a la lista de artistas.

Here is the caller graph for this function:

**3.10.3.6 getSongList()**

```
SongList & Spotify::getSongList ( ) [inline]
```

Obtiene la lista de canciones.

Returns

Referencia a la lista de canciones.

Here is the caller graph for this function:

**3.10.3.7 getUserList()**

```
UserList & Spotify::getUserList ( ) [inline]
```

Obtiene la lista de usuarios.

Returns

Referencia a la lista de usuarios.

3.10.3.8 leerArchivo()

```
string Spotify::leerArchivo (
    const string & nombre )
```

Lee el contenido de un archivo.

Parameters

<i>nombre</i>	Ruta del archivo a leer.
---------------	--------------------------

Returns

Contenido del archivo.

Here is the caller graph for this function:

**3.10.3.9 leerTemplateHtml()**

```
string Spotifyry::leerTemplateHtml (  
    string path )
```

Lee una plantilla HTML desde un archivo y devuelve su contenido como un string.

Parameters

<i>string</i>	Ruta del archivo de la plantilla HTML a leer.
---------------	---

Returns

Contenido de la plantilla HTML como un string.

Parameters

<i>path</i>	Ruta del archivo de la plantilla HTML a leer.
-------------	---

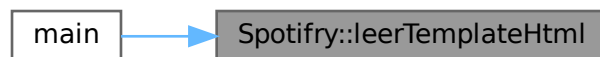
Returns

Contenido de la plantilla HTML como un string.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.10.3.10 numPlaylists()**

```
int Spotifry::numPlaylists (  
    string userEmail )
```

Genera un string HTML con la información de un álbum dado su título.

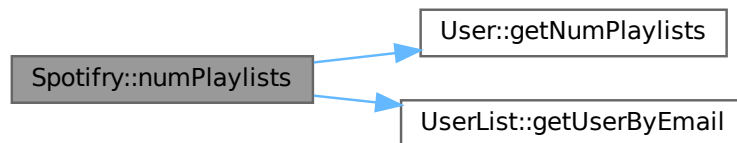
Parameters

<i>albumTitle</i>	El título del álbum para el cual se generará el HTML.
-------------------	---

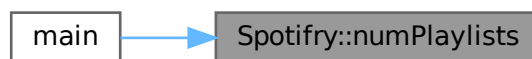
Returns

Un string HTML con la información del álbum, o un mensaje de error si el álbum no se encuentra.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.10.3.11 numSongs()**

```
int Spotify::numSongs (  
    string userEmail )
```

Obtiene el número de canciones guardadas por un usuario dado su email.

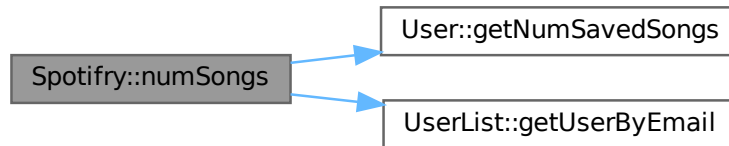
Parameters

<i>userEmail</i>	El email del usuario para el cual se obtendrá el número de canciones guardadas.
------------------	---

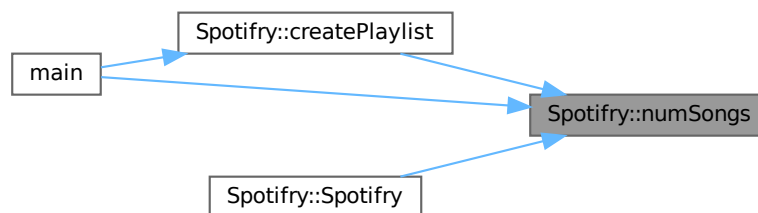
Returns

El número de canciones guardadas por el usuario, o -1 si el usuario no se encuentra.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.10.3.12 reemplazar()**

```

string Spotify::reemplazar (
    string html,
    const string & tag,
    const string & valor )
  
```

Reemplaza una etiqueta en un string HTML por un valor dado.

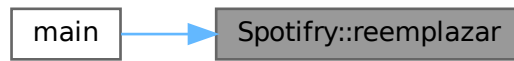
Parameters

<i>html</i>	El string HTML donde se realizará el reemplazo.
<i>tag</i>	La etiqueta a reemplazar (formato "{{TAG}}").
<i>valor</i>	El valor que reemplazará a la etiqueta en el string HTML.

Returns

El string HTML resultante después de realizar el reemplazo.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [include/spotify.h](#)
- [src/spotify.cpp](#)

3.11 User Class Reference

Representa a un usuario.

```
#include <user.h>
```

Collaboration diagram for User:

User
+ User() + User(string name, string email, Date birthdate, Gender gender) + User(const User &user) + ~User() + string getName() const + string getEmail() const + Date getBirthdate() const + string getGender() const + int getNumSavedSongs() const + int getNumPlaylists() const + void setName(string name) + void setEmail(string email) + void setBirthdate(Date birthdate) + void setGender(Gender gender) + User & operator=(const User &user) + Song & operator[](int i) const + Playlist & operator()(int i) const + const Song & operator()(int i, int j) const + bool addSong(Song *song_to_add) + bool addPlaylist(Playlist *playlist_to_add) + bool deleteSong(Song *song_to_delete) + bool deletePlaylist(Playlist *playlist_to_delete) + bool createPlaylist(Song **songs, int num_songs, string title)

Public Member Functions

- [User](#) ()
Constructor por defecto.
- [User](#) (string name, string email, [Date](#) birthdate, [Gender](#) gender)
Constructor con parámetros.
- [User](#) (const [User](#) &user)

- Constructor de copia.*

 - `~User ()`
- Destructor.*

 - `string getName () const`
Obtiene el nombre del usuario.
 - `string getEmail () const`
Obtiene el correo electrónico del usuario.
 - `Date getBirthdate () const`
Obtiene la fecha de nacimiento del usuario.
 - `string getGender () const`
Obtiene el género del usuario como string.
 - `int getNumSavedSongs () const`
Obtiene el número de canciones guardadas por el usuario.
 - `int getNumPlaylists () const`
Obtiene el número de playlists creadas por el usuario.
 - `void setName (string name)`
Establece el nombre del usuario.
 - `void setEmail (string email)`
Establece el correo electrónico del usuario.
 - `void setBirthdate (Date birthdate)`
Establece la fecha de nacimiento del usuario.
 - `void setGender (Gender gender)`
Establece el género del usuario.
 - `User & operator= (const User &user)`
Sobrecarga del operador de asignación.
 - `Song & operator[] (int i) const`
Sobrecarga del operador de indexación [] para acceder a las canciones guardadas por el usuario.
 - `Playlist & operator() (int i) const`
Sobrecarga del operador de indexación () para acceder a las playlists creadas por el usuario.
 - `const Song & operator() (int i, int j) const`
Sobrecarga del operador de indexación () para acceder a las canciones de una playlist específica.
 - `bool addSong (Song *song_to_add)`
Añade una canción a las canciones guardadas por el usuario si no está ya guardada.
 - `bool addPlaylist (Playlist *playlist_to_add)`
Añade una playlist a las playlists creadas por el usuario si no está ya agregada.
 - `bool deleteSong (Song *song_to_delete)`
Elimina una canción de las canciones guardadas por el usuario si está guardada.
 - `bool deletePlaylist (Playlist *playlist_to_delete)`
Elimina una playlist de las playlists creadas por el usuario si está creada.
 - `bool createPlaylist (Song **songs, int num_songs, string title)`
Crea una nueva playlist privada con las canciones dadas y la agrega a las playlists creadas por el usuario.

3.11.1 Detailed Description

Representa a un usuario.

3.11.2 Constructor & Destructor Documentation

3.11.2.1 User() [1/3]

```
User::User ( )
```

Constructor por defecto.

3.11.2.2 User() [2/3]

```
User::User (
    string name,
    string email,
    Date birthdate,
    Gender gender )
```

Constructor con parámetros.

Parameters

<i>name</i>	Nombre y apellidos del usuario.
<i>email</i>	Correo electrónico del usuario.
<i>birthdate</i>	Fecha de nacimiento del usuario.
<i>gender</i>	Género del usuario.

3.11.2.3 User() [3/3]

```
User::User (
    const User & user ) [inline]
```

Constructor de copia.

Parameters

<i>user</i>	El objeto User del cual se copiarán los valores.
-------------	--

3.11.2.4 ~User()

```
User::~~User ( )
```

Destructor.

Warning

Sólo libera la memoria de los arrays de punteros a canciones y playlists, no de las canciones ni playlists en sí, ya que el usuario no es el propietario de esos objetos.

3.11.3 Member Function Documentation

3.11.3.1 addPlaylist()

```
bool User::addPlaylist (
    Playlist * playlist_to_add )
```

Añade una playlist a las playlists creadas por el usuario si no está ya agregada.

Añade una playlist a las playlists creadas por el usuario si no está ya añadida.

Parameters

<i>playlist_to_add</i>	Puntero a la playlist que se añadirá.
------------------------	---------------------------------------

Returns

true si la playlist se añadió exitosamente, false si no se pudo agregar.

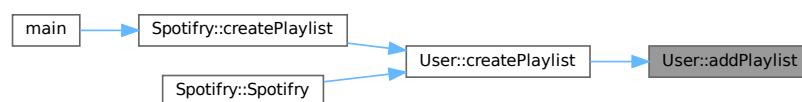
Parameters

<i>playlist_to_add</i>	Puntero a la playlist que se añadirá.
------------------------	---------------------------------------

Returns

true si la playlist se añadió exitosamente, false en caso contrario.

Here is the caller graph for this function:



3.11.3.2 addSong()

```
bool User::addSong (
    Song * song_to_add )
```

Añade una canción a las canciones guardadas por el usuario si no está ya guardada.

Parameters

<i>song_to_add</i>	Puntero a la canción que se añadirá.
--------------------	--------------------------------------

Returns

true si la canción se añadió exitosamente, false en caso contrario.

Here is the caller graph for this function:

**3.11.3.3 createPlaylist()**

```
bool User::createPlaylist (
    Song ** songs,
    int num_songs,
    string title )
```

Crea una nueva playlist privada con las canciones dadas y la agrega a las playlists creadas por el usuario.

Crea una nueva playlist privada con las canciones dadas y la añade a las playlists creadas por el usuario.

Parameters

<i>songs</i>	Array de punteros a las canciones que se agregarán a la nueva playlist.
<i>num_songs</i>	El número de canciones en el array <i>songs</i> .
<i>title</i>	El título de la nueva playlist.

Returns

true si la playlist se creó y agregó exitosamente, false si no se pudo crear o agregar la playlist.

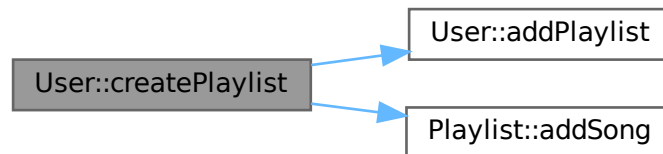
Parameters

<i>songs</i>	Array de punteros a las canciones que se añadirán a la nueva playlist.
<i>num_songs</i>	El número de canciones en el array <i>songs</i> .
<i>title</i>	El título de la nueva playlist.

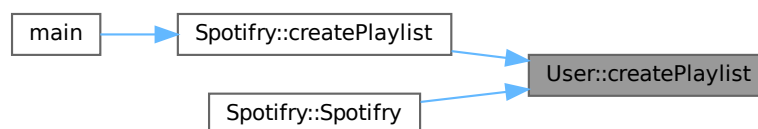
Returns

true si la playlist se creó y añadió exitosamente, false si no se pudo crear o añadir la playlist.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.11.3.4 deletePlaylist()**

```
bool User::deletePlaylist (
    Playlist * playlist_to_delete )
```

Elimina una playlist de las playlists creadas por el usuario si está creada.

Parameters

<code>playlist_to_delete</code>	Puntero a la playlist que se eliminará.
---------------------------------	---

Returns

true si la playlist se eliminó exitosamente, false si no se pudo eliminar.

Parameters

<code>playlist_to_delete</code>	Puntero a la playlist que se eliminará.
---------------------------------	---

Returns

true si la playlist se eliminó exitosamente, false en caso contrario.

3.11.3.5 deleteSong()

```
bool User::deleteSong (
    Song * song_to_delete )
```

Elimina una canción de las canciones guardadas por el usuario si está guardada.

Parameters

<code>song_to_delete</code>	Puntero a la canción que se eliminará.
-----------------------------	--

Returns

true si la canción se eliminó exitosamente, false en caso contrario.

3.11.3.6 getBirthdate()

```
Date User::getBirthdate ( ) const [inline]
```

Obtiene la fecha de nacimiento del usuario.

Returns

La fecha de nacimiento del usuario.

Here is the caller graph for this function:

**3.11.3.7 getEmail()**

```
string User::getEmail ( ) const [inline]
```

Obtiene el correo electrónico del usuario.

Returns

El correo electrónico del usuario.

Here is the caller graph for this function:

**3.11.3.8 getGender()**

```
string User::getGender ( ) const
```

Obtiene el género del usuario como string.

Returns

El género del usuario como string.

Here is the caller graph for this function:

**3.11.3.9 getName()**

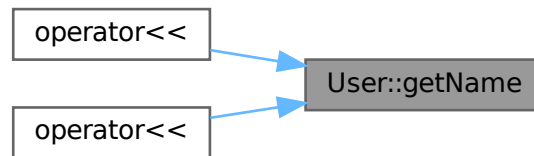
```
string User::getName ( ) const [inline]
```

Obtiene el nombre del usuario.

Returns

El nombre del usuario.

Here is the caller graph for this function:

**3.11.3.10 getNumPlaylists()**

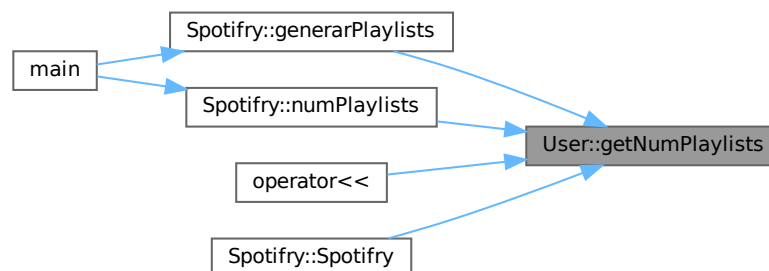
```
int User::getNumPlaylists ( ) const [inline]
```

Obtiene el número de playlists creadas por el usuario.

Returns

El número de playlists creadas por el usuario.

Here is the caller graph for this function:



3.11.3.11 `getNumSavedSongs()`

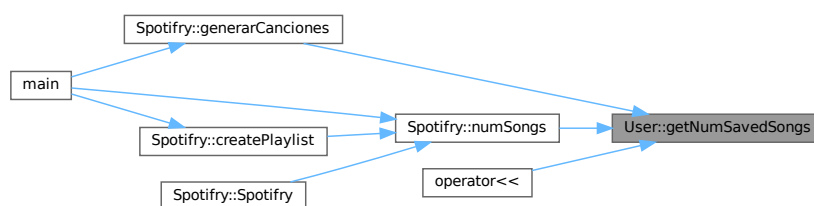
```
int User::getNumSavedSongs ( ) const [inline]
```

Obtiene el número de canciones guardadas por el usuario.

Returns

El número de canciones guardadas por el usuario.

Here is the caller graph for this function:



3.11.3.12 `operator()()` [1/2]

```
Playlist & User::operator() (
    int i ) const [inline]
```

Sobrecarga del operador de indexación `()` para acceder a las playlists creadas por el usuario.

Parameters

<i>i</i>	El índice de la playlist a acceder.
----------	-------------------------------------

Returns

Una referencia a la playlist en la posición dada del array `playlists`.

3.11.3.13 `operator()()` [2/2]

```
const Song & User::operator() (
    int i,
    int j ) const
```

Sobrecarga del operador de indexación `()` para acceder a las canciones de una playlist específica.

Parameters

<i>i</i>	El índice de la playlist a la que pertenece la canción.
<i>j</i>	El índice de la canción dentro de la playlist.

Returns

Una referencia a la canción en la posición dada dentro de la playlist dada.

3.11.3.14 operator=()

```
User & User::operator= (
    const User & user )
```

Sobrecarga del operador de asignación.

Parameters

<i>user</i>	El objeto User que se asignará al implícito.
-------------	--

Returns

Una referencia al objeto [User](#) resultante de la asignación.

3.11.3.15 operator[]()

```
Song & User::operator[] (
    int i ) const [inline]
```

Sobrecarga del operador de indexación `[]` para acceder a las canciones guardadas por el usuario.

Parameters

<i>i</i>	El índice de la canción a acceder.
----------	------------------------------------

Returns

Una referencia a la canción en la posición dada del array `saved_songs`.

3.11.3.16 setBirthdate()

```
void User::setBirthdate (
    Date birthdate ) [inline]
```

Establece la fecha de nacimiento del usuario.

Parameters

<i>birthdate</i>	La fecha de nacimiento del usuario.
------------------	-------------------------------------

3.11.3.17 setEmail()

```
void User::setEmail (
    string email ) [inline]
```

Establece el correo electrónico del usuario.

Parameters

<i>email</i>	El correo electrónico del usuario.
--------------	------------------------------------

3.11.3.18 setGender()

```
void User::setGender (
    Gender gender ) [inline]
```

Establece el género del usuario.

Parameters

<i>gender</i>	El género del usuario.
---------------	------------------------

3.11.3.19 setName()

```
void User::setName (
    string name ) [inline]
```

Establece el nombre del usuario.

Parameters

<i>name</i>	El nombre del usuario.
-------------	------------------------

The documentation for this class was generated from the following files:

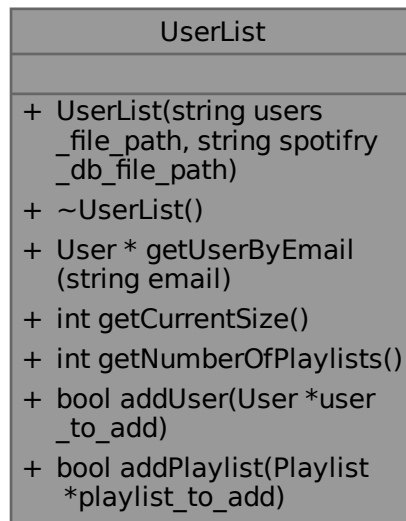
- [include/user.h](#)
- [src/user.cpp](#)

3.12 UserList Class Reference

Representa una lista de usuarios.

```
#include <userlist.h>
```

Collaboration diagram for UserList:



Public Member Functions

- [UserList](#) (string users_file_path, string spotify_db_file_path)
Constructor con parámetros.
- [~UserList](#) ()
Destructor.
- [User * getUserByEmail](#) (string email)
Obtiene un puntero al usuario con el email dado.
- [int getCurrentSize](#) ()
Obtiene el número actual de usuarios en la lista.
- [int getNumberOfPlaylists](#) ()
Obtiene el número total de playlists creadas por los usuarios.
- [bool addUser](#) ([User](#) *user_to_add)
Añade un usuario a la lista de usuarios si no se ha alcanzado el límite máximo de usuarios.
- [bool addPlaylist](#) ([Playlist](#) *playlist_to_add)
Añade una playlist a la lista de playlists si no se ha alcanzado el límite máximo de playlists.

3.12.1 Detailed Description

Representa una lista de usuarios.

3.12.2 Constructor & Destructor Documentation

3.12.2.1 `UserList()`

```
UserList::UserList (
    string users_file_path,
    string spotify_db_file_path )
```

Constructor con parámetros.

Parameters

<i>users_file_path</i>	Ruta del archivo .txt con los datos de los usuarios.
<i>spotify_db_file_path</i>	Ruta del archivo .txt con los datos de las canciones y playlists de los usuarios.

Here is the call graph for this function:



3.12.2.2 ~UserList()

```
UserList::~~UserList ( )
```

Destructor.

3.12.3 Member Function Documentation

3.12.3.1 addPlaylist()

```
bool UserList::addPlaylist (
    Playlist * playlist_to_add )
```

Añade una playlist a la lista de playlists si no se ha alcanzado el límite máximo de playlists.

Parameters

<i>playlist_to_add</i>	Un puntero a la playlist que se desea añadir.
------------------------	---

Returns

true si la playlist se añadió exitosamente, false en caso contrario.

3.12.3.2 addUser()

```
bool UserList::addUser (
    User * user_to_add )
```

Añade un usuario a la lista de usuarios si no se ha alcanzado el límite máximo de usuarios.

Parameters

<code>user_to_add</code>	Un puntero al usuario que se desea añadir.
--------------------------	--

Returns

true si el usuario se añadió exitosamente, false en caso contrario.

Here is the caller graph for this function:

**3.12.3.3 getCurrentSize()**

```
int UserList::getCurrentSize ( ) [inline]
```

Obtiene el número actual de usuarios en la lista.

Returns

El número actual de usuarios en la lista.

3.12.3.4 getNumberOfPlaylists()

```
int UserList::getNumberOfPlaylists ( ) [inline]
```

Obtiene el número total de playlists creadas por los usuarios.

Returns

El número total de playlists creadas por los usuarios.

3.12.3.5 getUserByEmail()

```
User * UserList::getUserByEmail (
    string email )
```

Obtiene un puntero al usuario con el email dado.

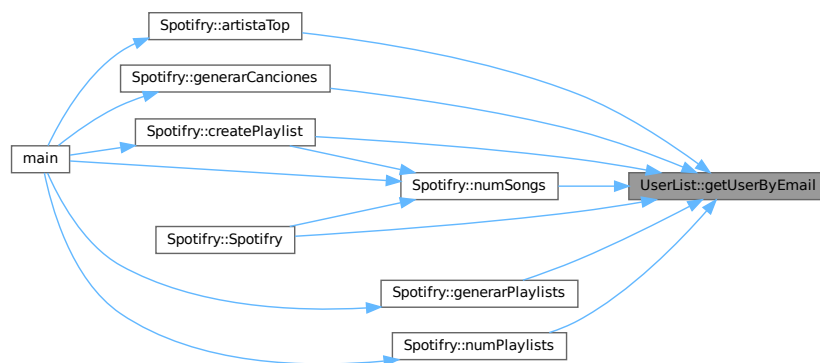
Parameters

<i>email</i>	El email del usuario a buscar.
--------------	--------------------------------

Returns

Un puntero al usuario con el email dado, o nullptr si no se encuentra ningún usuario con ese email.

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [include/userlist.h](#)
- [src/userlist.cpp](#)

Chapter 4

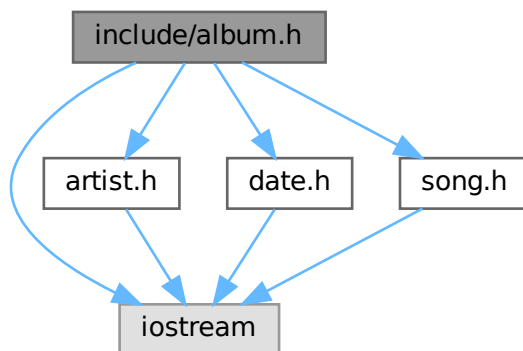
File Documentation

4.1 include/album.h File Reference

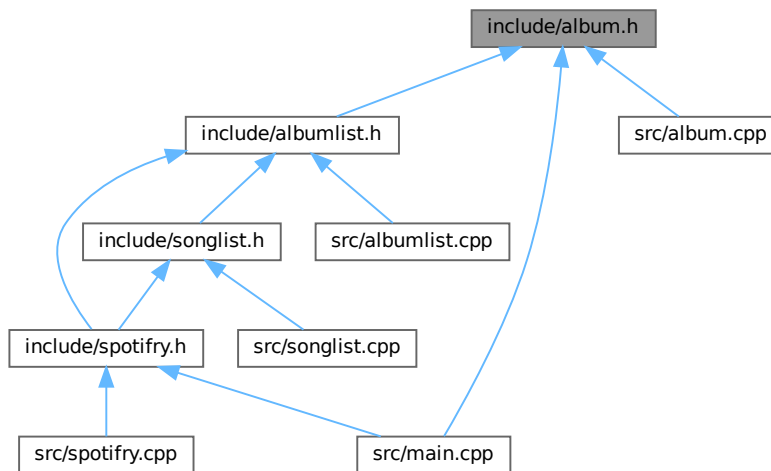
Declaración de la clase [Album](#).

```
#include <iostream>
#include "artist.h"
#include "date.h"
#include "song.h"
```

Include dependency graph for album.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Album](#)

Representa un álbum musical, con su título, artista, fecha de lanzamiento y canciones.

Functions

- ostream & [operator<<](#) (ostream &flujo, const [Album](#) &album)

Sobrecarga del operador de inserción para imprimir un álbum.

4.1.1 Detailed Description

Declaración de la clase [Album](#).

Author

Alonso Hernández Robles (F1)

4.1.2 Function Documentation

4.1.2.1 [operator<<\(\)](#)

```
ostream & operator<< (
    ostream & flujo,
    const Album & album )
```

Sobrecarga del operador de inserción para imprimir un álbum.

Parameters

<i>flujo</i>	Flujo de salida.
<i>album</i>	Álbum a imprimir.

Returns

Referencia al flujo de salida.

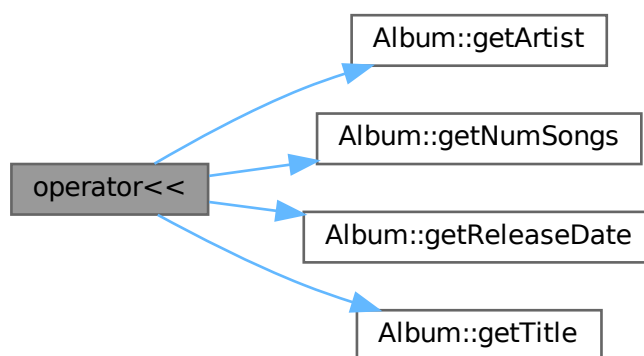
Parameters

<i>flujo</i>	Flujo de salida.
<i>album</i>	Álbum a imprimir.

Returns

Referencia al flujo de salida con la información del álbum.

Here is the call graph for this function:



4.2 album.h

[Go to the documentation of this file.](#)

```
00001
00007 #ifndef ALBUM_H
00008 #define ALBUM_H
00009
00010 #include <iostream>
00011 #include "artist.h"
00012 #include "date.h"
00013 #include "song.h"
00014
00015 using namespace std;
00016
00021 class Album {
```

```

00022
00023     private:
00024
00025         string title;
00026         Artist* artist;
00027         Date releaseDate;
00028         int num_songs;
00029         Song** songs;
00030
00035         void pasteAttributes(const Album& album);
00036
00037     public:
00038
00039         // CONSTRUCTORES Y DESTRUCTOR
00040
00044         Album();
00045
00054         Album(string title, Artist* artist, Date releaseDate, Song** songs, int num_songs);
00055
00060         Album(const Album& album) { this->pasteAttributes(album); }
00061
00066         ~Album();
00067
00068         // GETTERS
00069
00074         inline string getTitle() const { return this->title; }
00075
00080         inline Artist* getArtist() const { return this->artist; }
00081
00086         inline Date getReleaseDate() const { return this->releaseDate; }
00087
00092         inline int getNumSongs() const { return this->num_songs; }
00093
00099         inline Song* getSong(int i) const { return this->songs[i]; }
00100
00101         // SETTERS
00102
00107         inline void setTitle(string title) { this->title = title; }
00108
00113         inline void setArtist(Artist* artist) { this->artist = artist; }
00114
00119         inline void setReleaseDate(Date releaseDate) { this->releaseDate = releaseDate; }
00120
00126         void setNewSongs(int num_songs, Song** songs);
00127
00128         // SOBRECARGAS DE OPERADORES
00129
00135         Album& operator=(const Album& album);
00136
00142         inline const Song& operator[](int i) const { return *this->songs[i]; }
00143
00144         // OTROS MÉTODOS
00145
00151         bool addSong(Song* song);
00152
00153 };
00154
00161 ostream& operator<<(ostream& flujo, const Album& album);
00162
00163 #endif

```

4.3 include/albumlist.h File Reference

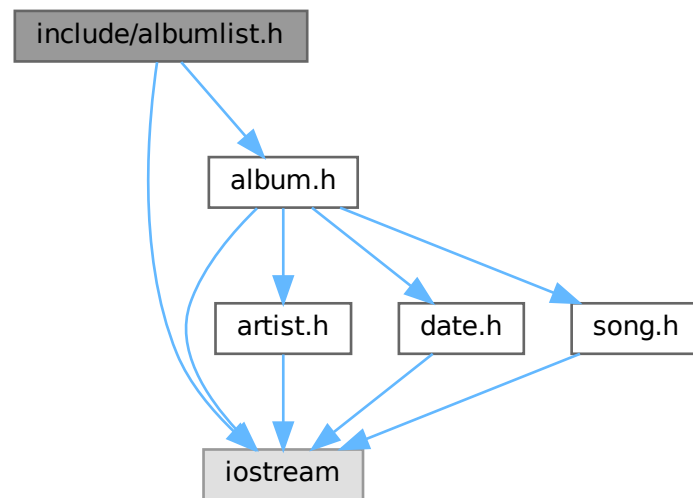
Declaración de la clase [AlbumList](#).

```

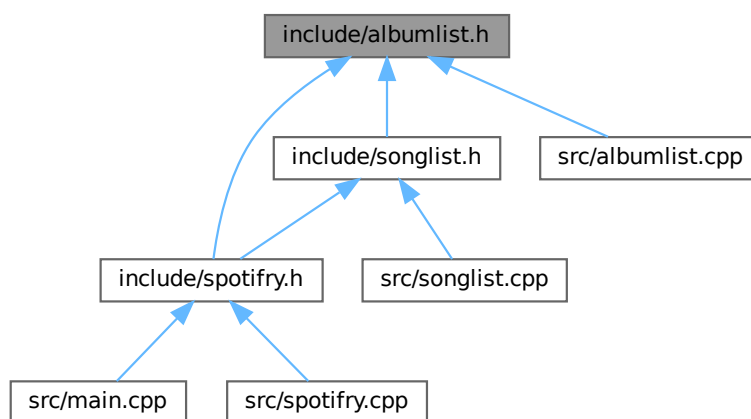
#include <iostream>
#include "album.h"

```


Include dependency graph for albumlist.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `AlbumList`
Representa una lista dinámica de álbumes.

Variables

- const int `MAX_ALBUMS` = 1000
Capacidad máxima de la lista de álbumes.

4.3.1 Detailed Description

Declaración de la clase [AlbumList](#).

Author

Alonso Hernández Robles (F1)

4.3.2 Variable Documentation

4.3.2.1 MAX_ALBUMS

```
const int MAX_ALBUMS = 1000
```

Capacidad máxima de la lista de álbumes.

4.4 albumlist.h

[Go to the documentation of this file.](#)

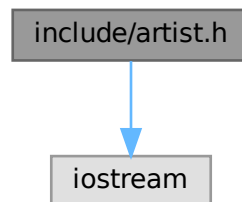
```
00001
00007 #ifndef ALBUMLIST_H
00008 #define ALBUMLIST_H
00009
00010 #include <iostream>
00011 #include "album.h"
00012
00013 using namespace std;
00014
00018 const int MAX_ALBUMS = 1000;
00019
00020 class SongList;
00021 class ArtistList;
00022
00027 class AlbumList{
00028
00029     private:
00030
00031         int current_size;
00032         Album* list[MAX_ALBUMS];
00033
00034     public:
00035
00036         // CONSTRUCTOR Y DESTRUCTOR
00037
00044         AlbumList(string file_path, ArtistList& artistList, SongList& songList);
00045
00049         ~AlbumList();
00050
00051         // GETTERS
00052
00057         inline int getCurrentSize() const { return this->current_size; }
00058
00064         inline Album* getAlbumById(int id) const { return this->list[id]; }
00065
00066         // OTROS MÉTODOS
00067
00073         bool dump(string file_path);
00074
00079         void printAlbum(string title) const;
00080
00081 };
00082
00083 #endif
```

4.5 include/artist.h File Reference

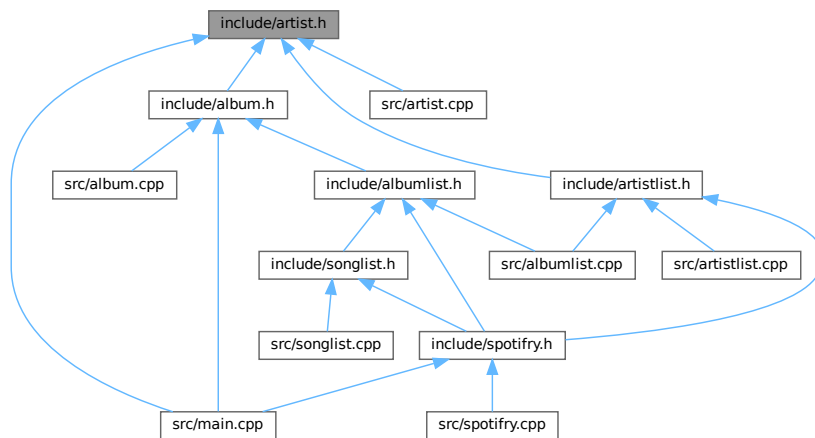
Declaración de la clase [Artist](#).

```
#include <iostream>
```

Include dependency graph for artist.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Artist](#)
Representa un artista con su nombre.

Functions

- ostream & [operator<<](#) (ostream &flujo, const [Artist](#) &artist)
Sobrecarga del operador de inserción para imprimir un artista.

4.5.1 Detailed Description

Declaración de la clase [Artist](#).

Author

Alonso Hernández Robles (F1)

4.5.2 Function Documentation

4.5.2.1 operator<<()

```
ostream & operator<< (
    ostream & flujo,
    const Artist & artist ) [inline]
```

Sobrecarga del operador de inserción para imprimir un artista.

Parameters

<i>flujo</i>	Flujo de salida.
<i>artist</i>	Artista a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:



4.6 artist.h

[Go to the documentation of this file.](#)

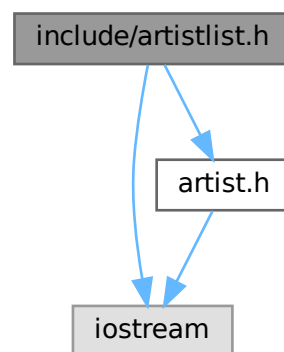
```
00001
00007 #ifndef ARTIST_H
00008 #define ARTIST_H
00009
00010 #include <iostream>
00011
00012 using namespace std;
00013
00018 class Artist {
00019     private:
00020
00021
```

```
00022     string name;
00023
00024     public:
00025
00026         // CONSTRUCTORES
00027
00031         Artist() { this->name = "New artist"; }
00032
00037         Artist(string name) { this->name = name; }
00038
00039         // GETTERS
00040
00045         inline string getName() const { return this->name; }
00046
00047         // SETTERS
00048
00053         inline void setName(string name) { this->name = name; }
00054
00055 };
00056
00063 inline ostream& operator<<(ostream& flujo, const Artist& artist) { return flujo << artist.getName(); }
00064
00065 #endif
```

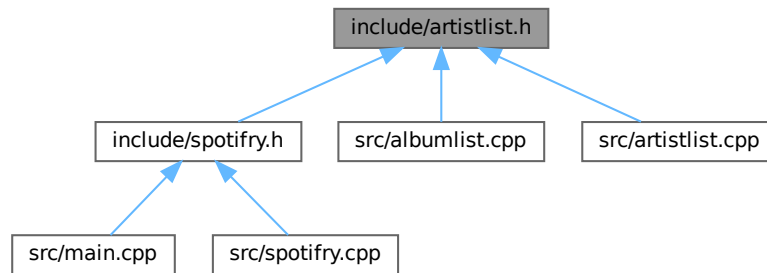
4.7 include/artistlist.h File Reference

Declaración de la clase [ArtistList](#).

```
#include <iostream>
#include "artist.h"
Include dependency graph for artistlist.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [ArtistList](#)
Representa una lista dinámica de artistas.

Variables

- const int [MAX_ARTISTS](#) = 1000
Capacidad máxima de la lista de artistas.

4.7.1 Detailed Description

Declaración de la clase [ArtistList](#).

Author

Alonso Hernández Robles (F1)

4.7.2 Variable Documentation

4.7.2.1 MAX_ARTISTS

```
const int MAX_ARTISTS = 1000
```

Capacidad máxima de la lista de artistas.

4.8 artistlist.h

[Go to the documentation of this file.](#)

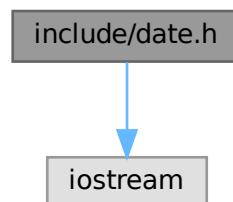
```
00001
00007 #ifndef ARTISTLIST_H
00008 #define ARTISTLIST_H
00009
00010 #include <iostream>
00011 #include "artist.h"
00012
00013 using namespace std;
00014
00018 const int MAX_ARTISTS = 1000;
00019
00024 class ArtistList{
00025     private:
00026         int current_size;
00029         Artist* list[MAX_ARTISTS];
00030
00031     public:
00032         // CONSTRUCTOR Y DESTRUCTOR
00034         ArtistList(string file_path);
00039         ~ArtistList();
00044         // GETTERS
00047         inline int getCurrentSize() const { return this->current_size; }
00052         inline Artist* getArtistById(int i) const { return this->list[i]; }
00060     };
00061 };
00062
00063 #endif
```

4.9 include/date.h File Reference

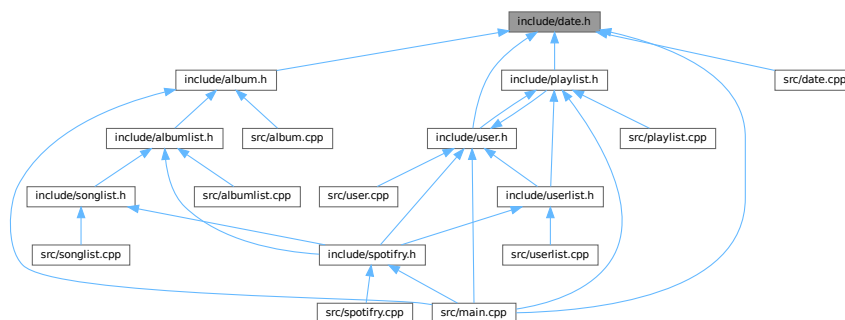
Declaración de la clase [Date](#).

```
#include <iostream>
```

Include dependency graph for date.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Date](#)

Representa una fecha con día, mes y año.

Functions

- ostream & [operator<<](#) (ostream &flujo, const [Date](#) &date)

Sobrecarga del operador de inserción para imprimir una fecha en formato dd/mm/yyyy.

4.9.1 Detailed Description

Declaración de la clase [Date](#).

Author

Alonso Hernández Robles (F1)

4.9.2 Function Documentation

4.9.2.1 operator<<()

```
ostream & operator<< (
    ostream & flujo,
    const Date & date ) [inline]
```

Sobrecarga del operador de inserción para imprimir una fecha en formato dd/mm/yyyy.

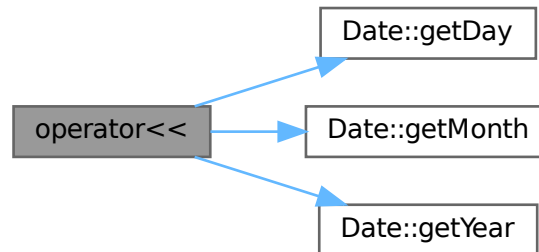
Parameters

<i>flujo</i>	Flujo de salida.
<i>date</i>	Fecha a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:



4.10 date.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef DATE_H
00008 #define DATE_H
00009
00010 #include <iostream>
00011
00012 using namespace std;
00013
00018 class Date {
00019     private:
00020         int day;
00021         int month;
00022         int year;
00023     public:
00024         // CONSTRUCTORES
00025         Date();
00026         Date(int day, int month, int year);
00027         inline int getDay() const { return this->day; }
00028         inline int getMonth() const { return this->month; }
00029         inline int getYear() const { return this->year; }
00030         inline void setDay(int day) { this->day = day; }
00031         inline void setMonth(int month) { this->month = month; }
00032         inline void setYear(int year) { this->year = year; }
00033         // SOBRECARGAS DE OPERADORES
00034         inline bool operator==(const Date& date) const {
00035             return this->day == date.day &&
00036                    this->month == date.month &&
00037                    this->year == date.year;
00038         }
00039         inline bool operator!=(const Date& date) const { return !(*this == date); }
00040
00041
00042
00043
00044
00045
00046
00047
00048
00049
00050
00051
00052
00053
00054
00055
00056
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081
00082
00083
00084
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
  
```

```

00104         bool operator<(const Date& date) const;
00105
00111         inline bool operator>(const Date& date) const { return date < *this; }
00112
00118         inline bool operator>=(const Date& date) const { return !(*this < date); }
00119
00125         inline bool operator<=(const Date& date) const { return !(date < *this); }
00126
00127     };
00128
00135     inline ostream& operator<<(ostream& flujo, const Date& date){
00136
00137         return flujo << (date.getDay() < 10 ? "0" : "")           // Mostrar 0 si d < 10
00138                        << date.getDay() << "/"                 // Día (-> dd)
00139                        << (date.getMonth() < 10 ? "0" : "")      // Mostrar 0 si m < 10
00140                        << date.getMonth() << "/"                // Mes (-> mm)
00141                        << (date.getYear() < 10 ? "0" : "")        // Mostrar 0 si y < 10
00142                        << (date.getYear() < 100 ? "0" : "")       // Mostrar 0 si y < 100
00143                        << (date.getYear() < 1000 ? "0" : "")     // Mostrar 0 si y < 1000
00144                        << date.getYear() << " "                 // Año (-> yyyy)
00145
00146     }
00147
00148 #endif

```

4.11 include/playlist.h File Reference

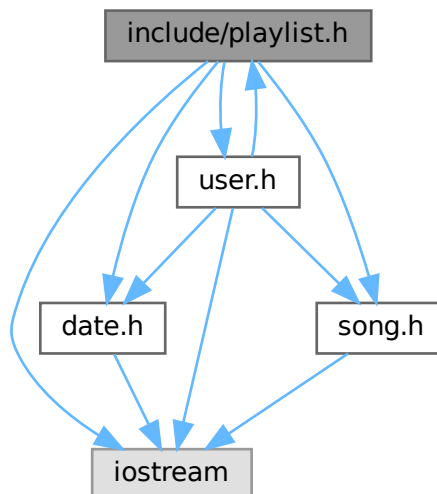
Declaración de la clase [Playlist](#).

```

#include <iostream>
#include "date.h"
#include "song.h"
#include "user.h"

```

Include dependency graph for playlist.h:



Enumerator

PUBLIC	
PRIVATE	

4.11.3 Function Documentation

4.11.3.1 operator<<()

```
ostream & operator<< (  
    ostream & flujo,  
    const Playlist & playlist )
```

Sobrecarga del operador de inserción para imprimir una playlist.

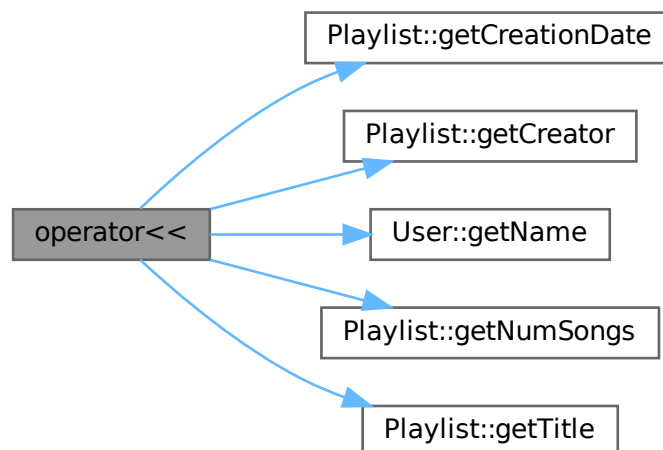
Parameters

<i>flujo</i>	Flujo de salida.
<i>playlist</i>	Playlist a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:



4.12 playlist.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef PLAYLIST_H
00008 #define PLAYLIST_H
00009
00010 #include <iostream>
00011 #include "date.h"
00012 #include "song.h"
00013 #include "user.h"
00014
00015 using namespace std;
00016
00020 enum PlaylistPrivacy {PUBLIC, PRIVATE};
00021
00022 class User;
00023
00028 class Playlist {
00029     private:
00030
00031         string title;
00032         User* creator;
00033         Date creationDate;
00034         int max_songs;
00035         int num_songs;
00036         Song** songs;
00037         PlaylistPrivacy privacy;
00038
00039     void init();
00040
00043     void pasteAttributes(const Playlist& playlist);
00044
00050
00051 public:
00052
00053     // CONSTRUCTORES Y DESTRUCTOR
00054
00055     Playlist();
00056
00059     Playlist(string title, User* creator, Date creationDate, PlaylistPrivacy privacy);
00060
00068     Playlist(const Playlist& playlist) { this->pasteAttributes(playlist); }
00069
00074     ~Playlist();
00075
00080
00081     // GETTERS
00082
00083     inline string getTitle() const { return this->title; }
00084
00088     inline const User& getCreator() const { return *this->creator; }
00089
00094     inline Date getCreationDate() const { return this->creationDate; }
00095
00100     inline int getMaxSongs() const { return this->max_songs; }
00101
00106     inline int getNumSongs() const { return this->num_songs; }
00107
00112     inline PlaylistPrivacy getPrivacy() const { return this->privacy; }
00113
00118
00119     // SETTERS
00120
00121     inline void setTitle(string title) { this->title = title; }
00122
00126     inline void setCreationDate(Date creationDate) { this->creationDate = creationDate; }
00127
00132     inline void setPrivacy(PlaylistPrivacy privacy) { this->privacy = privacy; }
00133
00138
00139     // OTROS MÉTODOS
00140
00141     bool addSong(Song* song, User* whoAddsIt);
00142
00148     bool deleteSong(Song* song, User* whoRemovesIt);
00149
00156
00157     // SOBRECARGAS DE OPERADORES
00158
00159     Playlist& operator=(const Playlist& playlist);
00160
00165     inline bool operator==(const Playlist& playlist) { return this->title == playlist.title &&
00166         this->creator == playlist.creator; }
00167
00172
00173     inline bool operator!=(const Playlist& playlist) { return !(*this == playlist); }
00174
00179
00180     inline const Song& operator[](int i) const { return *this->songs[i]; }
00181
00186
00187 };
00188
00189 ostream& operator<<(ostream& flujo, const Playlist& playlist);
00190
00197
00198 #endif

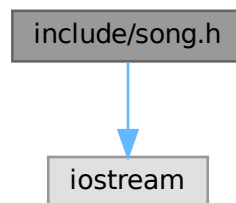
```

4.13 include/song.h File Reference

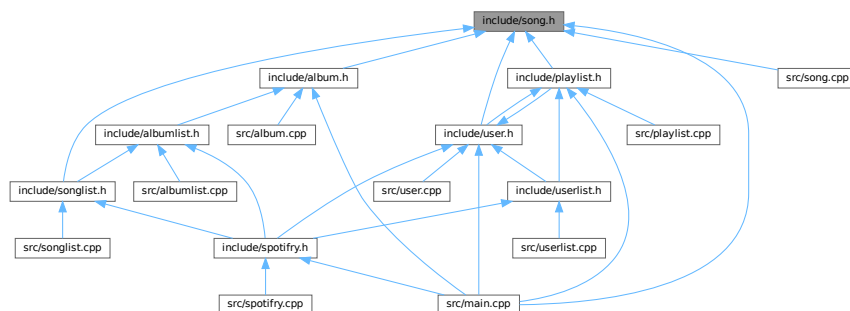
Declaración de la clase [Song](#).

```
#include <iostream>
```

Include dependency graph for song.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Song](#)
Representa una canción con un título, un género y una duración.

Enumerations

- enum [Genre](#) {
POP , ROCK , HIPHOP , COUNTRY ,
JAZZ , BLUES , ELECTRONIC , REGGAE ,
CLASSICAL , FOLK , METAL , UNKNOWN }
Enum para los géneros musicales.

Functions

- ostream & [operator<<](#) (ostream &flujo, const [Song](#) &song)
Sobrecarga del operador de inserción para imprimir una canción.

4.13.1 Detailed Description

Declaración de la clase [Song](#).

Author

Alonso Hernández Robles (F1)

4.13.2 Enumeration Type Documentation

4.13.2.1 Genre

enum [Genre](#)

Enum para los géneros musicales.

Enumerator

POP	
ROCK	
HIPHOP	
COUNTRY	
JAZZ	
BLUES	
ELECTRONIC	
REGGAE	
CLASSICAL	
FOLK	
METAL	
UNKNOWN	

4.13.3 Function Documentation

4.13.3.1 operator<<()

```
ostream & operator<< (  
    ostream & flujo,  
    const Song & song ) [inline]
```

Sobrecarga del operador de inserción para imprimir una canción.

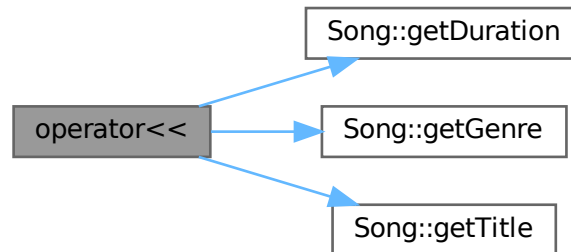
Parameters

<i>flujo</i>	Flujo de salida.
<i>song</i>	Canción a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:



4.14 song.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef SONG_H
00008 #define SONG_H
00009
00010 #include <iostream>
00011
00012 using namespace std;
00013
00017 enum Genre {POP, ROCK, HIPHOP, COUNTRY, JAZZ, BLUES, ELECTRONIC, REGGAE, CLASSICAL, FOLK, METAL,
UNKNOWN};
00018
00023 class Song {
00024     private:
00025         string title;
00026         Genre genre;
00027         int duration;
00028     public:
00029         // CONSTRUCTORES
00030         Song();
00031         Song(string title, Genre genre, int duration);
00032         // GETTERS
00033         inline string getTitle() const { return this->title; }
00034         string getGenre() const;
00035         inline int getDuration() const { return this->duration; }
00036         // SETTERS
00037         inline void setTitle(string title) { this->title = title; }
00038         inline void setGenre(Genre genre) { this->genre = genre; }
00039         inline void setDuration(int duration) { this->duration = duration; }
00040         // SOBRECARGAS DE OPERADORES
00041         inline bool operator==(const Song& song) const {

```



```

00096         return this->title == song.title &&
00097                this->genre == song.genre &&
00098                this->duration == song.duration;
00099     }
00105     inline bool operator!=(const Song& song) const { return !(*this == song); }
00106
00107 };
00108
00115 inline ostream& operator<<(ostream& flujo, const Song& song){
00116     return flujo << song.getTitle() << " (" <<
00117         << song.getDuration() / 60 << ":" << // Minutos
00118         << (song.getDuration() % 60 < 10 ? "0" : "") << // Mostrar 0 en decenas si seg < 10
00119         << song.getDuration() % 60 << ") | " << // Segundos
00120         << song.getGenre();
00121 }
00122
00123 #endif

```

4.15 include/songlist.h File Reference

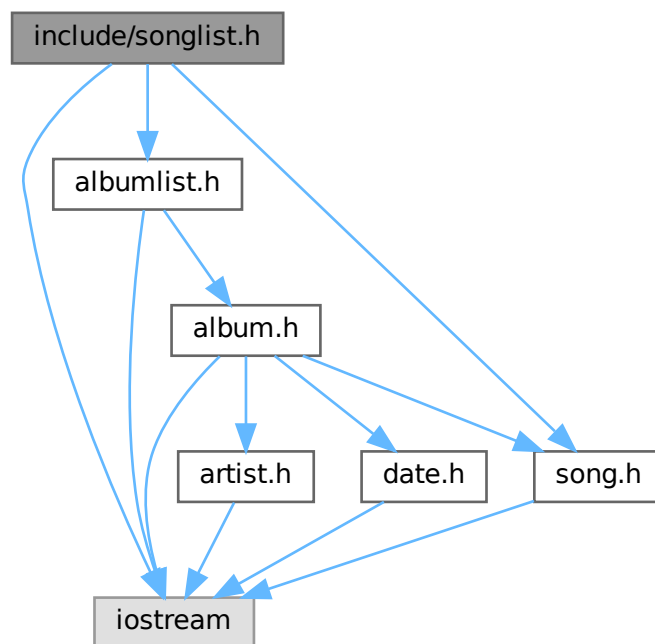
Declaración de la clase [SongList](#).

```

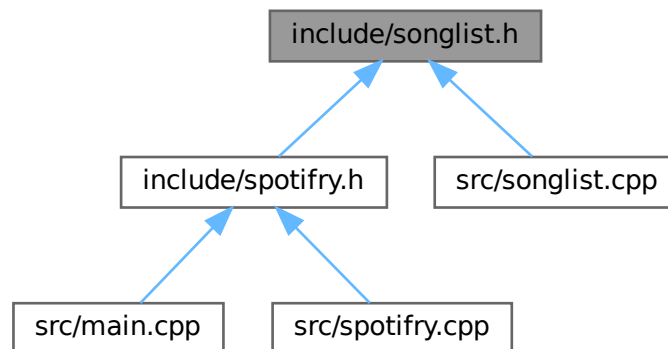
#include <iostream>
#include "albumlist.h"
#include "song.h"

```

Include dependency graph for songlist.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Block](#)
Estructura que representa un bloque de canciones en la lista de canciones.
- class [SongList](#)
Representa una lista dinámica de canciones utilizando bloques.

Variables

- const int [BLOCK_MAX_SIZE](#) = 10
Capacidad máxima de cada bloque de canciones.

4.15.1 Detailed Description

Declaración de la clase [SongList](#).

Author

Alonso Hernández Robles (F1)

4.15.2 Variable Documentation

4.15.2.1 BLOCK_MAX_SIZE

```
const int BLOCK_MAX_SIZE = 10
```

Capacidad máxima de cada bloque de canciones.

4.16 songlist.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef SONGLIST_H
00008 #define SONGLIST_H
00009
00010 #include <iostream>
00011 #include "albumlist.h"
00012 #include "song.h"
00013
00014 using namespace std;
00015
00019 const int BLOCK_MAX_SIZE = 10;
00020
00025 struct Block{
00026
00027     Song* list[BLOCK_MAX_SIZE];
00028     int current_size;
00029     Block* next;
00030
00031 };
00032
00033 class AlbumList;
00034
00039 class SongList{
00040
00041     private:
00042
00043         Block* firstBlock;
00044         Block* lastBlock;
00045
00046     public:
00047
00048         // CONSTRUCTOR Y DESTRUCTOR
00049
00055         SongList(string file_path, AlbumList& albumList);
00056
00060         ~SongList();
00061
00062         // OTROS MÉTODOS
00063
00073         bool addSong(string title, Genre genre, int duration, AlbumList &albumList, string album);
00074
00086         bool findSong(string searchTerm, Genre genre, int durationMin, int durationMax, Song**
results, int maxResults, int& numResults);
00087
00088 };
00089
00090 #endif

```

4.17 include/spotify.h File Reference

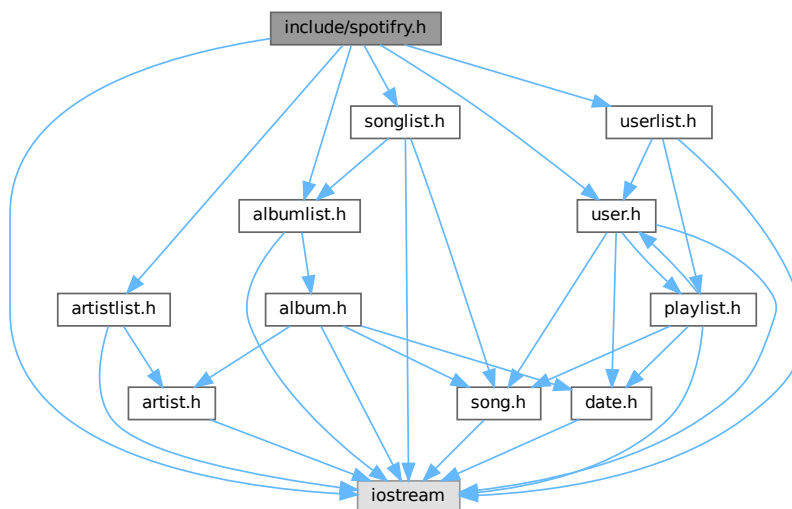
Declaración de la clase [Spotify](#).

```

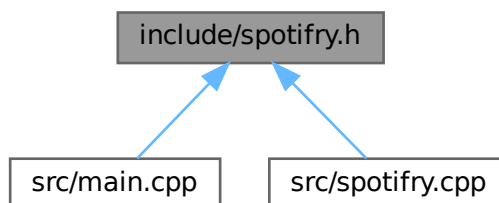
#include <iostream>
#include "albumlist.h"
#include "artistlist.h"
#include "songlist.h"
#include "user.h"
#include "userlist.h"

```

Include dependency graph for `spotify.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [Spotify](#)

Representa la aplicación principal de gestión de música, que contiene listas de artistas, álbumes, canciones y usuarios.

4.17.1 Detailed Description

Declaración de la clase [Spotify](#).

Author

Alonso Hernández Robles (F1)

4.18 spotifry.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef SPOTIFRY_H
00008 #define SPOTIFRY_H
00009
00010 #include <iostream>
00011 #include "albumlist.h"
00012 #include "artistlist.h"
00013 #include "songlist.h"
00014 #include "user.h"
00015 #include "userlist.h"
00016
00017 using namespace std;
00018
00023 class Spotifry{
00024
00025     private:
00026
00027         ArtistList artists;
00028         AlbumList albums;
00029         SongList songs;
00030         UserList users;
00031
00032     public:
00033
00034         // CONSTRUCTOR
00035
00043         Spotifry(string albums_file_path, string songs_file_path, string users_file_path, string
spotifry_db_file_path);
00044
00045         // GETTERS
00046
00051         inline SongList& getSongList() { return this->songs; }
00052
00057         inline AlbumList& getAlbumList() { return this->albums; }
00058
00063         inline UserList& getUserList() { return this->users; }
00064
00065         // OTROS MÉTODOS
00066
00075         bool createPlaylist(string user_email, string playlist_title, int num_songs, string*
song_names);
00076
00082         string leerArchivo(const string& nombre);
00083
00091         string reemplazar(string html, const string& tag, const string& valor);
00092
00098         string leerTemplateHtml(string string);
00099
00105         int numPlaylists(string userEmail);
00106
00112         int numSongs(string userEmail);
00113
00119         string artistaTop(string userEmail);
00120
00126         string generarPlaylists(string userEmail);
00127
00133         string generarCanciones(string userEmail);
00134
00135 };
00136
00137 #endif

```

4.19 include/user.h File Reference

Declaración de la clase [User](#).

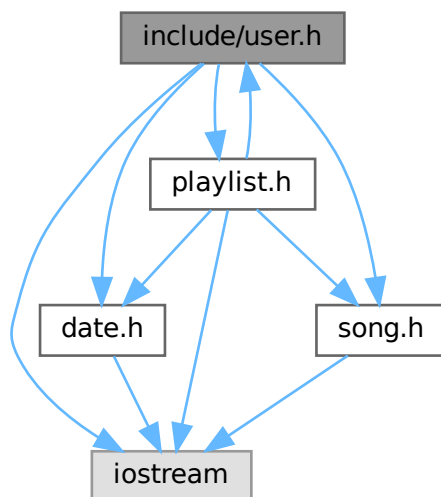
```

#include <iostream>
#include "date.h"
#include "playlist.h"

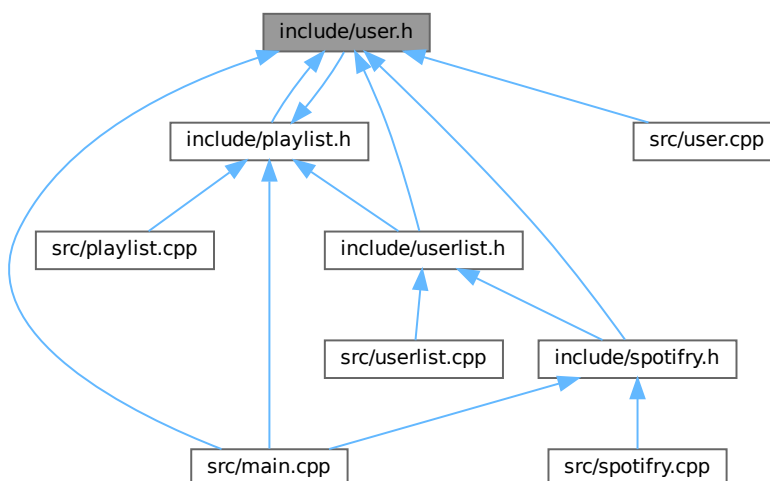
```

```
#include "song.h"
```

Include dependency graph for user.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [User](#)

Representa a un usuario.

Enumerations

- enum `Gender` { `FEMALE` , `MALE` , `OTHER` }
- Enum para representar el género de un usuario.*

Functions

- ostream & `operator<<` (ostream &flujo, const `User` &user)
- Sobrecarga del operador de inserción.*

4.19.1 Detailed Description

Declaración de la clase `User`.

Author

Alonso Hernández Robles (F1)

4.19.2 Enumeration Type Documentation

4.19.2.1 Gender

enum `Gender`

Enum para representar el género de un usuario.

Enumerator

FEMALE	
MALE	
OTHER	

4.19.3 Function Documentation

4.19.3.1 operator<<()

```
ostream & operator<< (  
    ostream & flujo,  
    const User & user ) [inline]
```

Sobrecarga del operador de inserción.

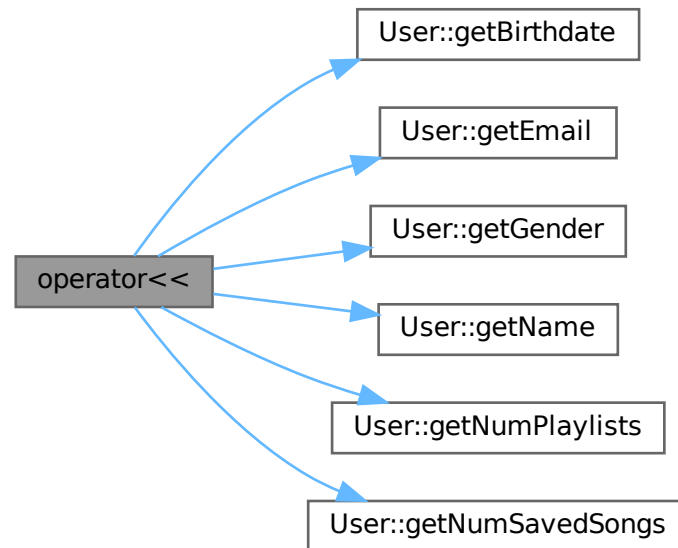
Parameters

<i>flujo</i>	Flujo de salida.
<i>user</i>	Usuario a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:



4.20 user.h

[Go to the documentation of this file.](#)

```

00001
00007 #ifndef USER_H
00008 #define USER_H
00009
00010 #include <iostream>
00011 #include "date.h"
00012 #include "playlist.h"
00013 #include "song.h"
00014
00015 using namespace std;
00016
00020 enum Gender {FEMALE, MALE, OTHER};
00021
00022 class Playlist;
00023
00027 class User {
00028     private:
00029
00030
00031         string name;
00032         string email;
00033         Date birthdate;
00034         Gender gender;
00035
00036         int max_saved_songs;
00037         int max_playlists;
00038
00039         int num_saved_songs;
00040         int num_playlists;
00041
00042         Song** saved_songs;
00043         Playlist** playlists;
  
```



```

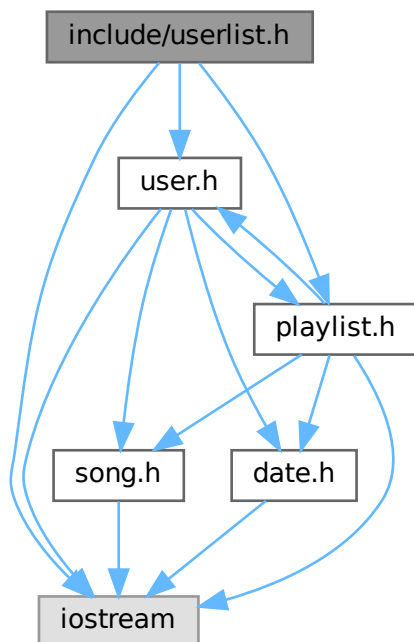
00044
00048     void init();
00049
00054     void pasteAttributes(const User& user);
00055
00056 public:
00057
00058     // CONSTRUCTORES Y DESTRUCTOR
00059
00063     User();
00064
00072     User(string name, string email, Date birthdate, Gender gender);
00073
00078     User(const User& user) { this->pasteAttributes(user); }
00079
00084     ~User();
00085
00086     // GETTERS
00087
00092     inline string getName() const { return this->name; }
00093
00098     inline string getEmail() const { return this->email; }
00099
00104     inline Date getBirthdate() const { return this->birthdate; }
00105
00110     string getGender() const;
00111
00116     inline int getNumSavedSongs() const { return this->num_saved_songs; }
00117
00122     inline int getNumPlaylists() const { return this->num_playlists; }
00123
00124     // SETTERS
00125
00130     inline void setName(string name) { this->name = name; }
00131
00136     inline void setEmail(string email) { this->email = email; }
00137
00142     inline void setBirthdate(Date birthdate) { this->birthdate = birthdate; }
00143
00148     inline void setGender(Gender gender) { this->gender = gender; }
00149
00150     // SOBRECARGAS DE OPERADORES
00151
00157     User& operator=(const User& user);
00158
00164     inline Song& operator[](int i) const { return *this->saved_songs[i]; }
00165
00171     inline Playlist& operator()(int i) const { return *this->playlists[i]; }
00172
00179     const Song& operator()(int i, int j) const;
00180
00181     // OTROS MÉTODOS
00182
00188     bool addSong(Song* song_to_add);
00189
00195     bool addPlaylist(Playlist* playlist_to_add);
00196
00202     bool deleteSong(Song* song_to_delete);
00203
00209     bool deletePlaylist(Playlist* playlist_to_delete);
00210
00218     bool createPlaylist(Song** songs, int num_songs, string title);
00219
00220 };
00221
00228 inline ostream& operator<<(ostream& flujo, const User& user){
00229
00230     return flujo << user.getName() << " ("
00231         << user.getEmail() << ") " << endl
00232         << user.getBirthdate() << " | "
00233         << user.getGender() << endl
00234         << user.getNumSavedSongs() << " saved songs | "
00235         << user.getNumPlaylists() << " created playlists" << endl;
00236
00237 }
00238
00239 #endif

```

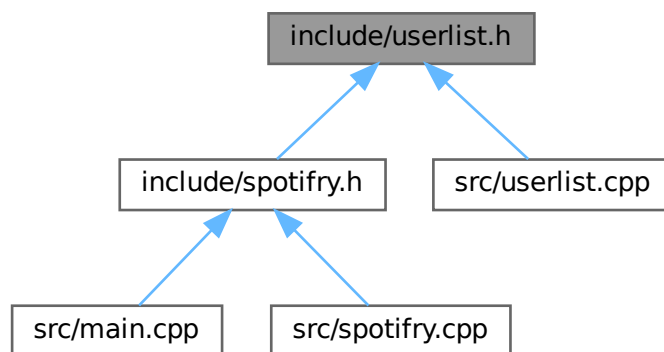
4.21 include/userlist.h File Reference

Declaración de la clase [UserList](#).

```
#include <iostream>
#include "user.h"
#include "playlist.h"
Include dependency graph for userlist.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [UserList](#)

Representa una lista de usuarios.

Variables

- const int `MAX_USERS` = 1000
Capacidad máxima de la lista de usuarios.
- const int `MAX_PLAYLISTS` = 2000
Capacidad máxima de la lista de playlists.

4.21.1 Detailed Description

Declaración de la clase `UserList`.

Author

Alonso Hernández Robles (F1)

4.21.2 Variable Documentation

4.21.2.1 MAX_PLAYLISTS

```
const int MAX_PLAYLISTS = 2000
```

Capacidad máxima de la lista de playlists.

4.21.2.2 MAX_USERS

```
const int MAX_USERS = 1000
```

Capacidad máxima de la lista de usuarios.

4.22 userlist.h

[Go to the documentation of this file.](#)

```
00001
00007 #ifndef USERLIST_H
00008 #define USERLIST_H
00009
00010 #include <iostream>
00011 #include "user.h"
00012 #include "playlist.h"
00013
00014 using namespace std;
00015
00019 const int MAX_USERS = 1000;
00020
00024 const int MAX_PLAYLISTS = 2000;
00025
00029 class UserList{
00030
00031     private:
00032
00033         int current_size;
00034         User* list[MAX_USERS];
00035
```

```

00036         int number_of_playlists;
00037         Playlist* playlists[MAX_PLAYLISTS];
00038
00039     public:
00040
00041         // CONSTRUCTOR Y DESTRUCTOR
00042
00043         UserList(string users_file_path, string spotify_db_file_path);
00044
00045         ~UserList();
00046
00047         // GETTERS
00048
00049         User* getUserByEmail(string email);
00050
00051         inline int getCurrentSize() { return this->current_size; }
00052
00053         inline int getNumberOfPlaylists() { return this->number_of_playlists; }
00054
00055         // OTROS MÉTODOS
00056
00057         bool addUser(User* user_to_add);
00058
00059         bool addPlaylist(Playlist* playlist_to_add);
00060
00061     };
00062 #endif

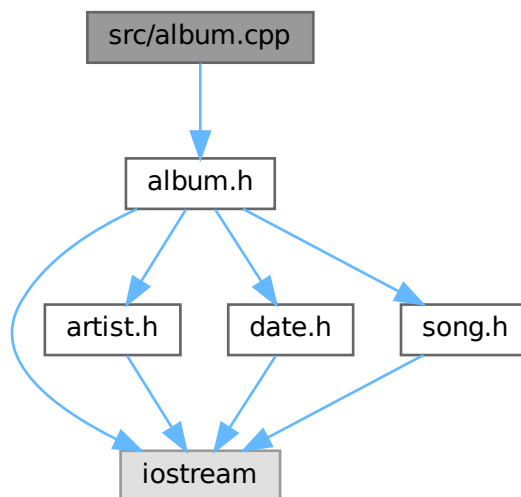
```

4.23 src/album.cpp File Reference

Implementación de la clase [Album](#).

```
#include "album.h"
```

Include dependency graph for album.cpp:



Functions

- ostream & [operator<<](#) (ostream &flujo, const [Album](#) &album)
Sobrecarga del operador de inserción para imprimir un álbum.

4.23.1 Detailed Description

Implementación de la clase [Album](#).

Author

Alonso Hernández Robles (F1)

4.23.2 Function Documentation

4.23.2.1 `operator<<()`

```
ostream & operator<< (
    ostream & flujo,
    const Album & album )
```

Sobrecarga del operador de inserción para imprimir un álbum.

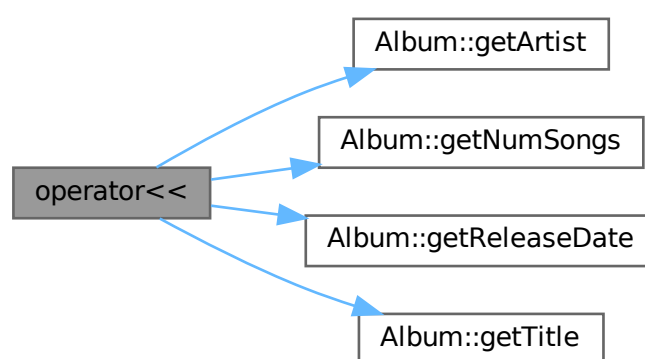
Parameters

<i>flujo</i>	Flujo de salida.
<i>album</i>	Álbum a imprimir.

Returns

Referencia al flujo de salida con la información del álbum.

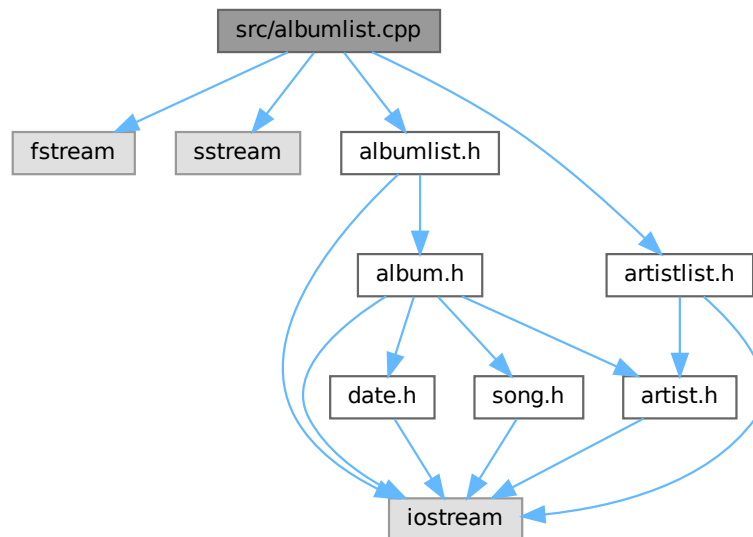
Here is the call graph for this function:



4.24 src/albumlist.cpp File Reference

Implementación de la clase [AlbumList](#).

```
#include <fstream>
#include <sstream>
#include "albumlist.h"
#include "artistlist.h"
Include dependency graph for albumlist.cpp:
```



4.24.1 Detailed Description

Implementación de la clase [AlbumList](#).

Author

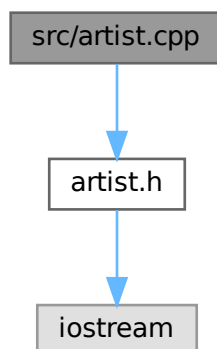
Alonso Hernández Robles (F1)

4.25 src/artist.cpp File Reference

Implementación de la clase [Artist](#).

```
#include "artist.h"
```

Include dependency graph for artist.cpp:



4.25.1 Detailed Description

Implementación de la clase [Artist](#).

Author

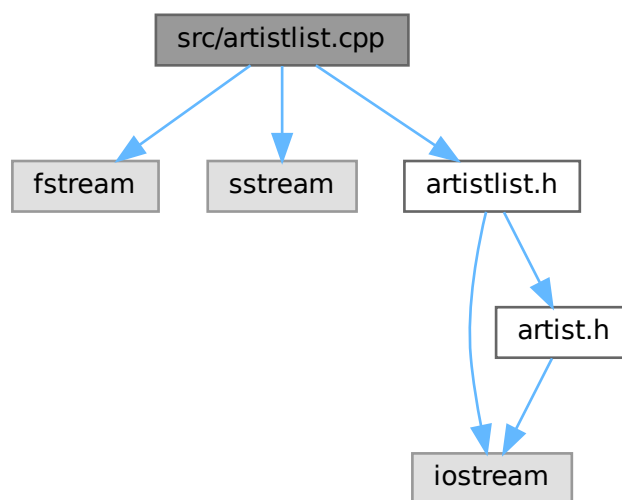
Alonso Hernández Robles (F1)

4.26 src/artistlist.cpp File Reference

Implementación de la clase [ArtistList](#).

```
#include <fstream>
#include <sstream>
#include "artistlist.h"
```

Include dependency graph for `artistlist.cpp`:



4.26.1 Detailed Description

Implementación de la clase [ArtistList](#).

Author

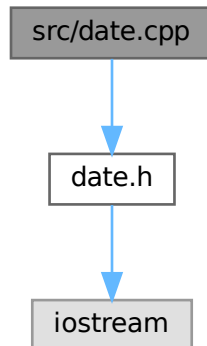
Alonso Hernández Robles (F1)

4.27 `src/date.cpp` File Reference

Implementación de la clase [Date](#).


```
#include "date.h"
```

Include dependency graph for date.cpp:



4.27.1 Detailed Description

Implementación de la clase [Date](#).

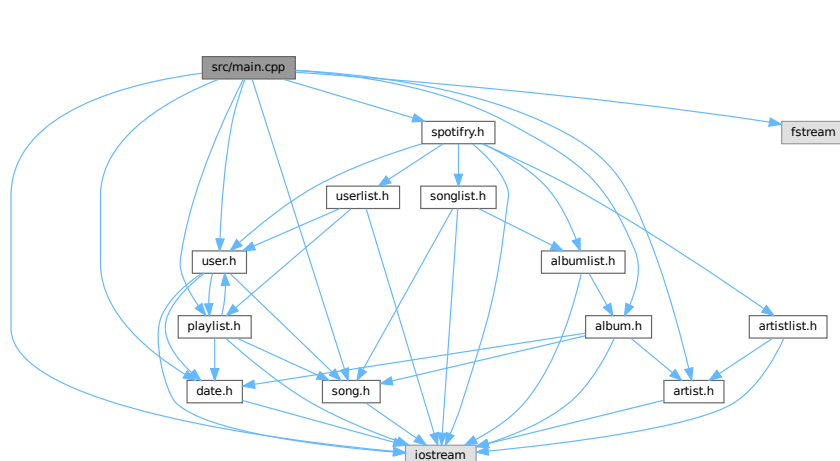
Author

Alonso Hernández Robles (F1)

4.28 src/main.cpp File Reference

```
#include <iostream>
#include "artist.h"
#include "album.h"
#include "song.h"
#include "date.h"
#include "user.h"
#include "playlist.h"
#include "spotify.h"
#include <fstream>
```

Include dependency graph for main.cpp:



Functions

- int [main](#) (int argc, char *argv[])

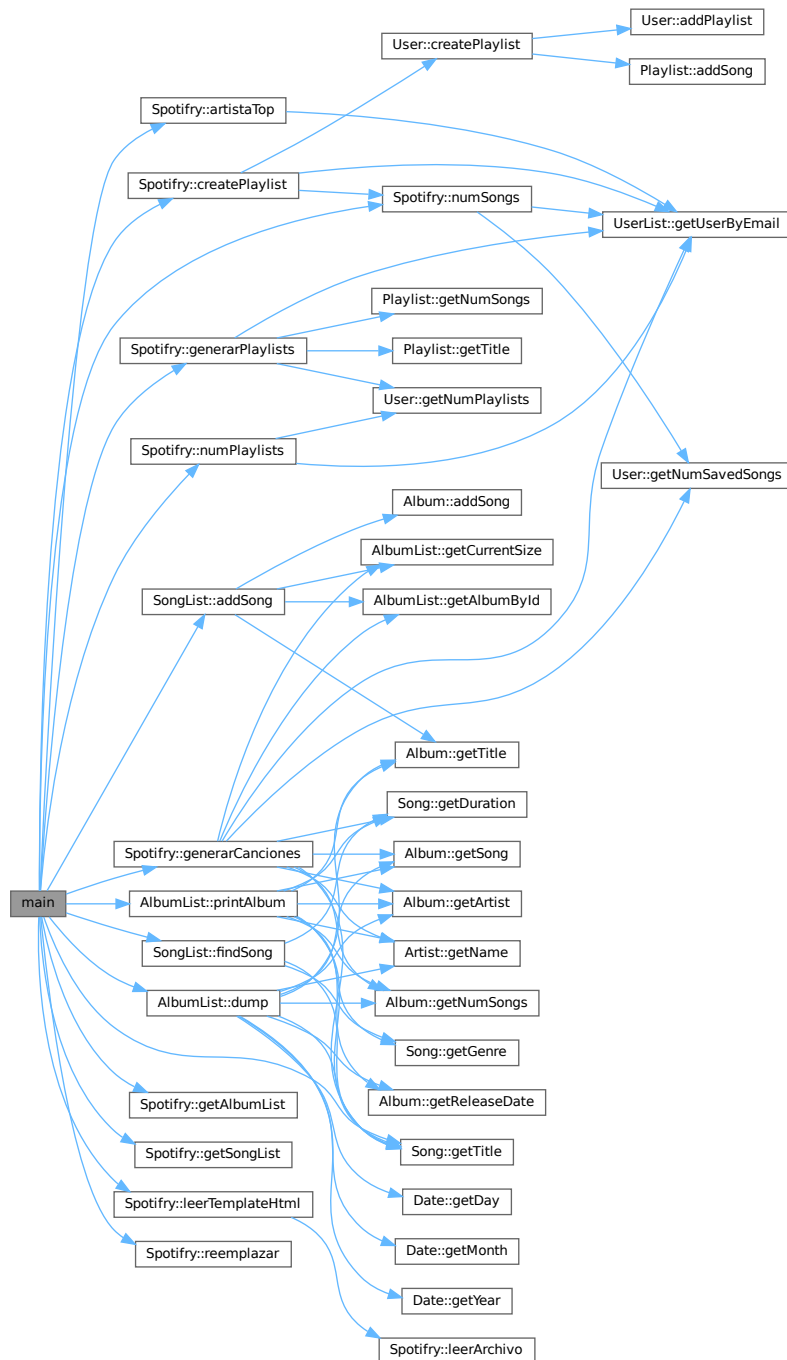
4.28.1 Function Documentation

4.28.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

TEST DE INTEGRACIÓN PARTE 2: "SPOTIFRY PERSISTENCIA Y BÚSQUEDA" Here is the call graph for this

function:

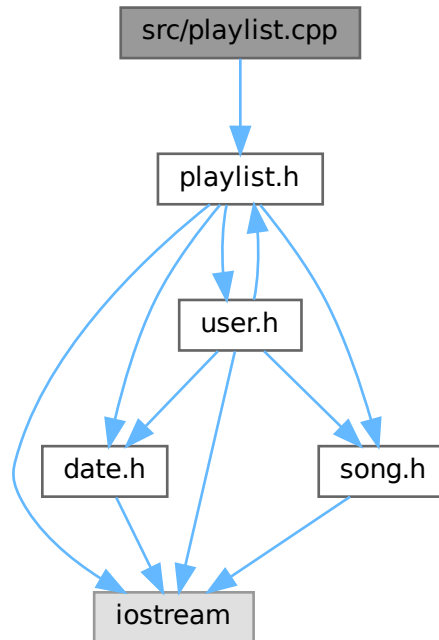


4.29 src/playlist.cpp File Reference

Implementación de la clase [Playlist](#).

```
#include "playlist.h"
```

Include dependency graph for playlist.cpp:



Functions

- ostream & `operator<<` (ostream &flujo, const `Playlist` &playlist)
Sobrecarga del operador de inserción para imprimir una playlist.

4.29.1 Detailed Description

Implementación de la clase `Playlist`.

Author

Alonso Hernández Robles (F1)

4.29.2 Function Documentation

4.29.2.1 `operator<<()`

```
ostream & operator<< (  
    ostream & flujo,  
    const Playlist & playlist )
```

Sobrecarga del operador de inserción para imprimir una playlist.

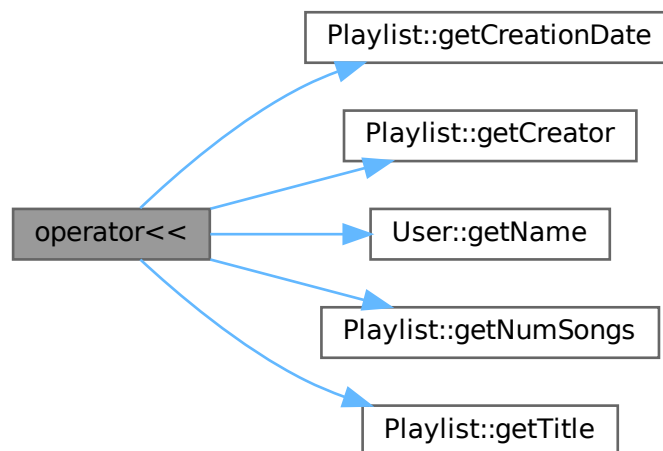
Parameters

<i>flujo</i>	Flujo de salida.
<i>playlist</i>	Playlist a imprimir.

Returns

Referencia al flujo de salida.

Here is the call graph for this function:

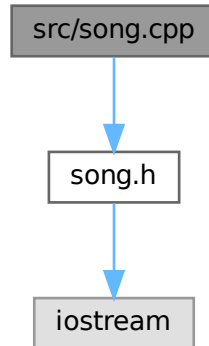


4.30 src/song.cpp File Reference

Implementación de la clase [Song](#).

```
#include "song.h"
```

Include dependency graph for song.cpp:



4.30.1 Detailed Description

Implementación de la clase [Song](#).

Author

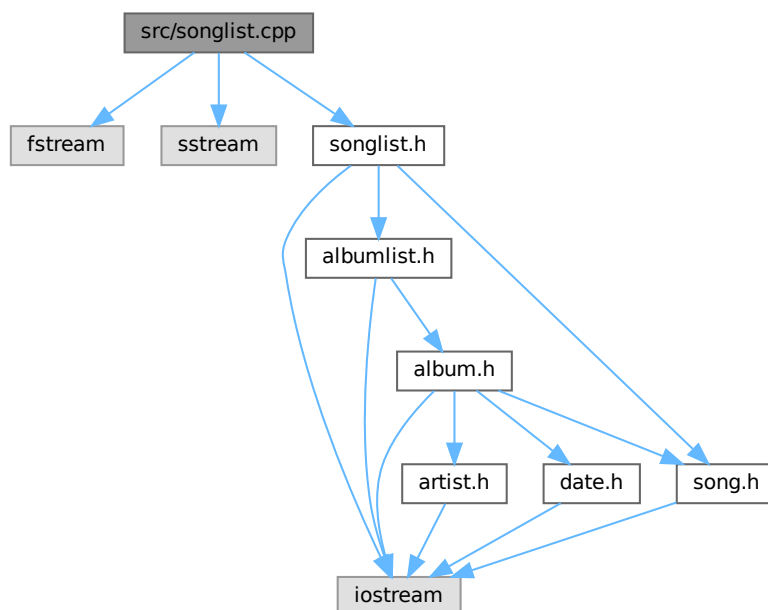
Alonso Hernández Robles (F1)

4.31 src/songlist.cpp File Reference

Implementación de la clase [SongList](#).

```
#include <fstream>
#include <sstream>
#include "songlist.h"
```

Include dependency graph for songlist.cpp:



4.31.1 Detailed Description

Implementación de la clase [SongList](#).

Author

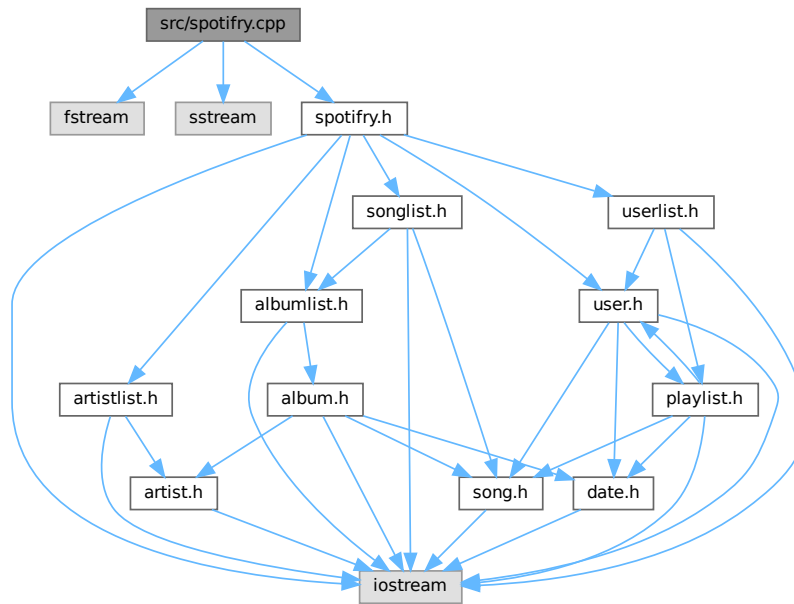
Alonso Hernández Robles (F1)

4.32 src/spotify.cpp File Reference

Implementación de la clase [Spotify](#).

```
#include <fstream>
#include <sstream>
#include "spotify.h"
```

Include dependency graph for `spotifyfry.cpp`:



4.32.1 Detailed Description

Implementación de la clase [Spotifyfry](#).

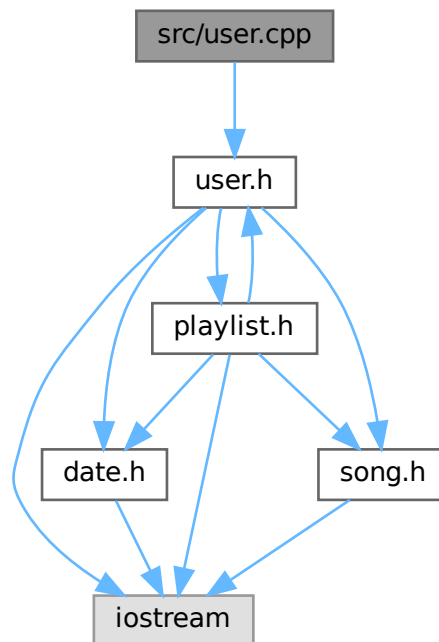
Author

Alonso Hernández Robles (F1)

4.33 src/user.cpp File Reference

```
#include "user.h"
```


Include dependency graph for user.cpp:

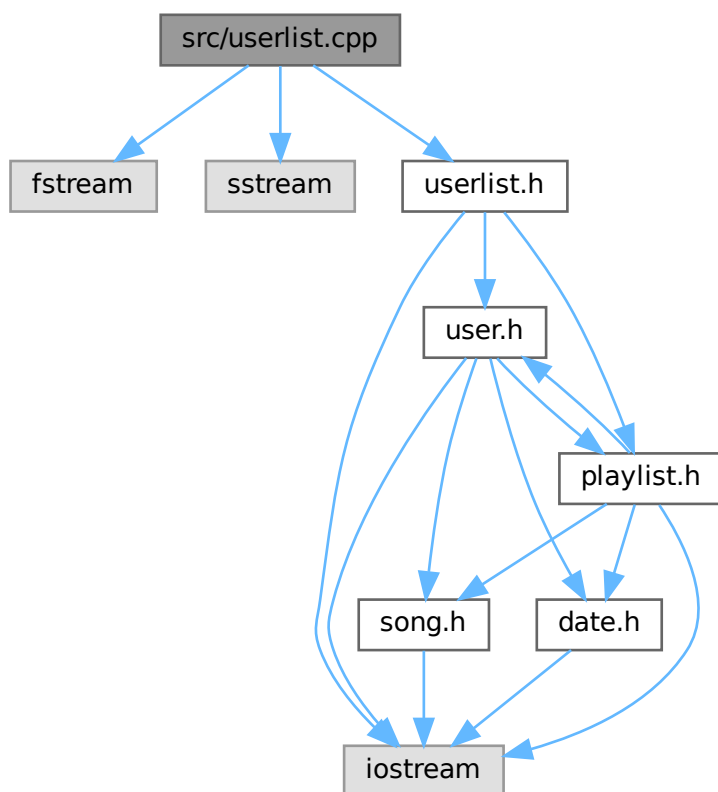


4.34 src/userlist.cpp File Reference

Implementación de la clase [UserList](#).

```
#include <fstream>
#include <sstream>
#include "userlist.h"
```

Include dependency graph for `userlist.cpp`:



4.34.1 Detailed Description

Implementación de la clase [UserList](#).

Author

Alonso Hernández Robles (F1)

Index

- ~Album
 - Album, [8](#)
- ~AlbumList
 - AlbumList, [15](#)
- ~ArtistList
 - ArtistList, [22](#)
- ~Playlist
 - Playlist, [35](#)
- ~SongList
 - SongList, [48](#)
- ~User
 - User, [67](#)
- ~UserList
 - UserList, [79](#)
- addPlaylist
 - User, [68](#)
 - UserList, [79](#)
- addSong
 - Album, [8](#)
 - Playlist, [35](#)
 - SongList, [48](#)
 - User, [68](#)
- addUser
 - UserList, [79](#)
- Album, [5](#)
 - ~Album, [8](#)
 - addSong, [8](#)
 - Album, [7](#), [8](#)
 - getArtist, [9](#)
 - getNumSongs, [9](#)
 - getReleaseDate, [10](#)
 - getSong, [10](#)
 - getTitle, [11](#)
 - operator=, [11](#)
 - operator[], [12](#)
 - setArtist, [12](#)
 - setNewSongs, [12](#)
 - setReleaseDate, [13](#)
 - setTitle, [13](#)
- album.cpp
 - operator<<, [115](#)
- album.h
 - operator<<, [84](#)
- AlbumList, [13](#)
 - ~AlbumList, [15](#)
 - AlbumList, [14](#)
 - dump, [15](#)
 - getAlbumByld, [16](#)
 - getCurrentSize, [17](#)
 - printAlbum, [17](#)
- albumlist.h
 - MAX_ALBUMS, [88](#)
- Artist, [19](#)
 - Artist, [19](#)
 - getName, [20](#)
 - setName, [20](#)
- artist.h
 - operator<<, [90](#)
- artistaTop
 - Spotify, [54](#)
- ArtistList, [21](#)
 - ~ArtistList, [22](#)
 - ArtistList, [21](#)
 - getArtistByld, [22](#)
 - getCurrentSize, [22](#)
- artistlist.h
 - MAX_ARTISTS, [92](#)
- Block, [23](#)
 - current_size, [25](#)
 - list, [25](#)
 - next, [25](#)
- BLOCK_MAX_SIZE
 - songlist.h, [104](#)
- BLUES
 - song.h, [101](#)
- CLASSICAL
 - song.h, [101](#)
- COUNTRY
 - song.h, [101](#)
- createPlaylist
 - Spotify, [55](#)
 - User, [69](#)
- current_size
 - Block, [25](#)
- Date, [25](#)
 - Date, [27](#)
 - getDay, [28](#)
 - getMonth, [28](#)
 - getYear, [28](#)
 - operator!=, [29](#)
 - operator<, [29](#)
 - operator<=, [30](#)
 - operator>, [30](#)
 - operator>=, [31](#)
 - operator==, [30](#)
 - setDay, [31](#)

- setMonth, [31](#)
 - setYear, [31](#)
- date.h
 - operator<<, [94](#)
- deletePlaylist
 - User, [70](#)
- deleteSong
 - Playlist, [36](#)
 - User, [71](#)
- dump
 - AlbumList, [15](#)
- ELECTRONIC
 - song.h, [101](#)
- FEMALE
 - user.h, [109](#)
- findSong
 - SongList, [49](#)
- FOLK
 - song.h, [101](#)
- Gender
 - user.h, [109](#)
- generarCanciones
 - Spotify, [56](#)
- generarPlaylists
 - Spotify, [57](#)
- Genre
 - song.h, [101](#)
- getAlbumByld
 - AlbumList, [16](#)
- getAlbumList
 - Spotify, [58](#)
- getArtist
 - Album, [9](#)
- getArtistByld
 - ArtistList, [22](#)
- getBirthdate
 - User, [71](#)
- getCreationDate
 - Playlist, [36](#)
- getCreator
 - Playlist, [37](#)
- getCurrentSize
 - AlbumList, [17](#)
 - ArtistList, [22](#)
 - UserList, [80](#)
- getDay
 - Date, [28](#)
- getDuration
 - Song, [43](#)
- getEmail
 - User, [71](#)
- getGender
 - User, [72](#)
- getGenre
 - Song, [44](#)
- getMaxSongs
 - Playlist, [37](#)
- getMonth
 - Date, [28](#)
- getName
 - Artist, [20](#)
 - User, [72](#)
- getNumberOfPlaylists
 - UserList, [80](#)
- getNumPlaylists
 - User, [73](#)
- getNumSavedSongs
 - User, [73](#)
- getNumSongs
 - Album, [9](#)
 - Playlist, [37](#)
- getPrivacy
 - Playlist, [38](#)
- getReleaseDate
 - Album, [10](#)
- getSong
 - Album, [10](#)
- getSongList
 - Spotify, [59](#)
- getTitle
 - Album, [11](#)
 - Playlist, [38](#)
 - Song, [44](#)
- getUserByEmail
 - UserList, [80](#)
- getUserList
 - Spotify, [59](#)
- getYear
 - Date, [28](#)
- HIPHOP
 - song.h, [101](#)
- include/album.h, [83](#), [85](#)
- include/albumlist.h, [86](#), [88](#)
- include/artist.h, [89](#), [90](#)
- include/artistlist.h, [91](#), [93](#)
- include/date.h, [93](#), [95](#)
- include/playlist.h, [96](#), [98](#)
- include/song.h, [100](#), [102](#)
- include/songlist.h, [103](#), [105](#)
- include/spotify.h, [105](#), [107](#)
- include/user.h, [107](#), [110](#)
- include/userlist.h, [111](#), [113](#)
- JAZZ
 - song.h, [101](#)
- leerArchivo
 - Spotify, [59](#)
- leerTemplateHtml
 - Spotify, [60](#)
- list
 - Block, [25](#)
- main

- main.cpp, 120
- main.cpp
 - main, 120
- MALE
 - user.h, 109
- MAX_ALBUMS
 - albumlist.h, 88
- MAX_ARTISTS
 - artistlist.h, 92
- MAX_PLAYLISTS
 - userlist.h, 113
- MAX_USERS
 - userlist.h, 113
- METAL
 - song.h, 101
- next
 - Block, 25
- numPlaylists
 - Spotify, 61
- numSongs
 - Spotify, 62
- operator!=
 - Date, 29
 - Playlist, 38
 - Song, 45
- operator<
 - Date, 29
- operator<<
 - album.cpp, 115
 - album.h, 84
 - artist.h, 90
 - date.h, 94
 - playlist.cpp, 122
 - playlist.h, 98
 - song.h, 101
 - user.h, 109
- operator<=
 - Date, 30
- operator>
 - Date, 30
- operator>=
 - Date, 31
- operator()
 - User, 74
- operator=
 - Album, 11
 - Playlist, 39
 - User, 75
- operator==
 - Date, 30
 - Playlist, 39
 - Song, 45
- operator[]
 - Album, 12
 - Playlist, 39
 - User, 75
- OTHER
 - user.h, 109
- Playlist, 32
 - ~Playlist, 35
 - addSong, 35
 - deleteSong, 36
 - getCreationDate, 36
 - getCreator, 37
 - getMaxSongs, 37
 - getNumSongs, 37
 - getPrivacy, 38
 - getTitle, 38
 - operator!=, 38
 - operator=, 39
 - operator==, 39
 - operator[], 39
 - Playlist, 34, 35
 - setCreationDate, 40
 - setPrivacy, 40
 - setTitle, 41
- playlist.cpp
 - operator<<, 122
- playlist.h
 - operator<<, 98
 - PlaylistPrivacy, 97
 - PRIVATE, 98
 - PUBLIC, 98
- PlaylistPrivacy
 - playlist.h, 97
- POP
 - song.h, 101
- printAlbum
 - AlbumList, 17
- PRIVATE
 - playlist.h, 98
- PUBLIC
 - playlist.h, 98
- reemplazar
 - Spotify, 63
- REGGAE
 - song.h, 101
- ROCK
 - song.h, 101
- setArtist
 - Album, 12
- setBirthdate
 - User, 75
- setCreationDate
 - Playlist, 40
- setDay
 - Date, 31
- setDuration
 - Song, 46
- setEmail
 - User, 75
- setGender
 - User, 76

- setGenre
 - Song, 46
- setMonth
 - Date, 31
- setName
 - Artist, 20
 - User, 76
- setNewSongs
 - Album, 12
- setPrivacy
 - Playlist, 40
- setReleaseDate
 - Album, 13
- setTitle
 - Album, 13
 - Playlist, 41
 - Song, 46
- setYear
 - Date, 31
- Song, 41
 - getDuration, 43
 - getGenre, 44
 - getTitle, 44
 - operator!=, 45
 - operator==, 45
 - setDuration, 46
 - setGenre, 46
 - setTitle, 46
 - Song, 43
- song.h
 - BLUES, 101
 - CLASSICAL, 101
 - COUNTRY, 101
 - ELECTRONIC, 101
 - FOLK, 101
 - Genre, 101
 - HIPHOP, 101
 - JAZZ, 101
 - METAL, 101
 - operator<<, 101
 - POP, 101
 - REGGAE, 101
 - ROCK, 101
 - UNKNOWN, 101
- SongList, 46
 - ~SongList, 48
 - addSong, 48
 - findSong, 49
 - SongList, 47
- songlist.h
 - BLOCK_MAX_SIZE, 104
- Spotify, 51
 - artistaTop, 54
 - createPlaylist, 55
 - generarCanciones, 56
 - generarPlaylists, 57
 - getAlbumList, 58
 - getSongList, 59
 - getUserList, 59
 - leerArchivo, 59
 - leerTemplateHtml, 60
 - numPlaylists, 61
 - numSongs, 62
 - reemplazar, 63
 - Spotify, 53
- src/album.cpp, 114
- src/albumlist.cpp, 115
- src/artist.cpp, 116
- src/artistlist.cpp, 117
- src/date.cpp, 118
- src/main.cpp, 119
- src/playlist.cpp, 121
- src/song.cpp, 123
- src/songlist.cpp, 124
- src/spotify.cpp, 125
- src/user.cpp, 126
- src/userlist.cpp, 127
- UNKNOWN
 - song.h, 101
- User, 64
 - ~User, 67
 - addPlaylist, 68
 - addSong, 68
 - createPlaylist, 69
 - deletePlaylist, 70
 - deleteSong, 71
 - getBirthdate, 71
 - getEmail, 71
 - getGender, 72
 - getName, 72
 - getNumPlaylists, 73
 - getNumSavedSongs, 73
 - operator(), 74
 - operator=, 75
 - operator[], 75
 - setBirthdate, 75
 - setEmail, 75
 - setGender, 76
 - setName, 76
 - User, 67
- user.h
 - FEMALE, 109
 - Gender, 109
 - MALE, 109
 - operator<<, 109
 - OTHER, 109
- UserList, 76
 - ~UserList, 79
 - addPlaylist, 79
 - addUser, 79
 - getCurrentSize, 80
 - getNumberOfPlaylists, 80
 - getUserByEmail, 80
 - UserList, 78
- userlist.h
 - MAX_PLAYLISTS, 113

MAX_USERS, [113](#)